DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE)
FACULTY OF SCIENCE & ENGINEERING

Object Detection and Navigation Assistance for Visually Impaired Individuals

A Real-Time Assistive System Using YOLO11 and Bilingual Voice Guidance

Submitted by

MD Nahiduzzaman Dipu

ID: 223002150

B.Sc. in Computer Science & Engineering (Day)

Supervised by

[Supervisor Name — to be supplied]

[Designation — to be supplied]

Department of Computer Science & Engineering

[University Name — to be supplied]

[Submission Date — to be supplied]

Declaration

I declare that this thesis, entitled *"Object Detection and Navigation Assistance for Visually Impaired Individuals"*, is the result of my own work carried out under the supervision of [Supervisor Name], Department of Computer Science & Engineering. The system described in this report — including the dataset preparation pipeline, the annotation workflow, the trained detection model, the navigation logic, the bilingual voice subsystem and the web interface — was designed, implemented and evaluated by me.

Material drawn from other sources is cited in the text and listed in the References. Where existing software libraries and pre-trained models were used, this is stated explicitly in Chapters 3 and 4, and the boundary between those components and my own contribution is identified. All experimental values reported in Chapter 5 were measured on the hardware described in Section 4.1; no result has been estimated or reproduced from other work.

This thesis has not been submitted previously, in whole or in part, for any other degree or qualification.

_____

MD Nahiduzzaman Dipu
ID: 223002150
Date: [to be supplied]

Approval / Certification

This is to certify that the thesis entitled *"Object Detection and Navigation Assistance for Visually Impaired Individuals"*, submitted by **MD Nahiduzzaman Dipu** (ID: 223002150), has been carried out under my supervision. The work is original and, to the best of my knowledge, has not been submitted elsewhere for the award of any degree. It is recommended for acceptance in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science & Engineering.

_____	_____
Supervisor	**Head of the Department**
[Name, Designation]	[Name]
Department of CSE	Department of CSE
_____	_____
Internal Examiner	**External Examiner**
[Name]	[Name]

Acknowledgement

I am grateful to my supervisor, [Supervisor Name], for guidance throughout this work, and in particular for the advice to narrow the class list to obstacles that actually change a pedestrian's decision rather than trying to detect everything visible in a street scene. That constraint shaped the design more than any other single decision.

I thank the Department of Computer Science & Engineering for access to laboratory facilities, and my classmates for testing the interface and pointing out that the early version spoke far too often to be tolerable in practice — an observation that led directly to the announcement-suppression logic described in Section 3.11.

Finally, I thank my family for their support during the data collection and annotation phases, which took considerably longer than I had planned.

Abstract

Independent mobility remains a persistent difficulty for people with visual impairment, particularly on the footpaths of dense South Asian cities where the walking surface is frequently obstructed by parked vehicles, poles, street vendors and damaged pavement. The white cane detects obstacles only on contact and conveys nothing about what an obstacle is, while camera-based assistive systems reported in the literature are largely trained on Western urban imagery that does not represent this environment.

This thesis presents VisionAssist AI, a real-time assistive prototype that detects obstacles from a single monocular camera and converts them into spoken navigation guidance in Bengali and English. A dataset of 2,787 Dhaka footpath images was assembled from an initial pool of 21,359 unlabelled images, de-duplicated by perceptual hashing, and annotated for nine obstacle classes through a machine-assisted workflow in which an open-vocabulary detector proposed candidate boxes for human review. A YOLO11n detector was fine-tuned on the resulting 2,782 annotated images for 100 epochs on an NVIDIA RTX 3050 Ti.

Detections are converted into guidance by a deterministic pipeline rather than a second learned model: bounding-box geometry yields a horizontal zone (left, centre, right) and an apparent-proximity band, a priority function ranks obstacles, and an event-based announcement policy with a configurable cooldown decides when speech is warranted. Guidance sentences are constructed by a template grammar that handles Bengali subject–object–verb order and animacy-dependent numeral classifiers, and are spoken through a neural text-to-speech voice. The same pipeline serves three input modes: live camera, uploaded image, and uploaded recorded video.

On a held-out test split of 276 images the model achieves mAP@0.5 of 0.419 and mAP@0.5:0.95 of 0.297, with precision 0.645 and recall 0.404. Performance is strongly class-dependent: `person` (AP@0.5 = 0.837) and `parked_vehicle` (0.781) are detected reliably, whereas `pothole` and `open_drain` obtain zero recall. The cause is identified as annotation scarcity rather than model capacity — these classes had 17 and 12 training instances respectively, because the open-vocabulary annotation aid could not propose candidates for ground-surface defects. End-to-end throughput is 22.9 FPS in live camera mode and 13.7 FPS in video mode.

The contribution of this work is threefold: a reproducible machine-assisted annotation pipeline for an unlabelled Bangladeshi footpath corpus; an explainable, non-learned navigation layer whose behaviour can be audited line by line; and an empirical demonstration that open-vocabulary annotation transfers well to common object categories but fails on the ground-surface hazards that matter most for this application. The system is an assistive prototype and is not proposed as a sole navigation aid.

Keywords: Object Detection; YOLO11; Computer Vision; Navigation Assistance; Visually Impaired; Deep Learning; Bengali Speech Guidance; Assistive Technology.

Table of Contents

Declaration ii

Approval / Certification iii

Acknowledgement iv

Abstract v

List of Figures / Tables / Abbreviations vii

Chapter 1 — Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Motivation 2

1.4 Research Gap 3

1.5 Aim · 1.6 Objectives · 1.7 Research Questions 3

1.8 Scope · 1.9 Contributions · 1.10 Organization 4

Chapter 2 — Literature Review 6

2.1–2.9 Background areas 6

2.10 Comparative Analysis 10

Chapter 3 — Methodology 11

3.1–3.3 System overview, architecture, input sources 11

3.4–3.7 Dataset, collection, annotation, preprocessing 13

3.8–3.9 Detection model and training 16

3.10–3.12 Position, navigation, voice 18

3.13–3.15 Video processing, workflow, algorithms 21

Chapter 4 — System Implementation 24

Chapter 5 — Experiments and Results 28

Chapter 6 — Discussion 35

Chapter 7 — Conclusion and Future Work 38

References 40

Appendices A–I 42

Page numbers are indicative of document order. They should be regenerated after any edit to the source file, since this document is produced by an HTML-to-PDF pipeline rather than a system with automatic cross-referencing.

List of Figures

Fig. 3.1 — Overall system architecture 11
Fig. 3.2 — Data processing and annotation pipeline 14
Fig. 3.3 — Object detection workflow (dual-model fusion) 17
Fig. 3.4 — Horizontal zone partitioning 18
Fig. 3.5 — Navigation decision workflow 20
Fig. 3.6 — Live camera workflow 21
Fig. 3.7 — Image processing workflow 21
Fig. 3.8 — Recorded video workflow 22
Fig. 3.9 — Voice guidance workflow 22
Fig. 3.10 — Overall system flowchart 23
Fig. 5.1 — Training and validation curves 29
Fig. 5.2 — Normalized confusion matrix 30
Fig. 5.3 — Precision–recall curve 31
Fig. 5.4 — F1–confidence curve 31
Fig. 5.5 — Per-class mAP 32
Fig. 5.6 — Detection example (indoor) 32
Fig. 5.7 — Live camera frame with overlay 33
Fig. 5.8 — Interface, image mode 33
Fig. 5.9 — Interface, video mode 34

List of Tables

Table 2.1 — Comparison with related systems 10
Table 3.1 — Class list and Bengali labels 13
Table 3.2 — Dataset reduction pipeline 14
Table 3.3 — Split sizes and per-class instances 15
Table 3.4 — Detection model configuration 16
Table 3.5 — Training configuration 17
Table 3.6 — Priority score components 19
Table 4.1 — Hardware and software environment 24
Table 4.2 — Software modules 25
Table 5.1 — Overall detection results 29
Table 5.2 — Per-class performance 30
Table 5.3 — Measured runtime performance by input mode 33
Table 5.4 — Bengali and English guidance examples 34
Table 6.1 — Summary of limitations 37

List of Abbreviations

AP	Average Precision
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
FPS	Frames Per Second
IoU	Intersection over Union
mAP	mean Average Precision
MJPEG	Motion JPEG
NMS	Non-Maximum Suppression
SOV	Subject–Object–Verb (word order)
TTS	Text-to-Speech
VIP	Visually Impaired Person
YOLO	You Only Look Once

CHAPTER 1

Introduction

1.1 Background

Sight supplies most of the information a pedestrian uses to move safely: what lies ahead, how far away it is, and whether the surface can be trusted. When that channel is absent, the same information must be recovered some other way. The World Health Organization estimates that at least 2.2 billion people live with a near or distance vision impairment [1], and mobility is consistently among the functions most affected.

The white cane remains the dominant aid, and for good reason: it is inexpensive, needs no power, and gives immediate tactile feedback. Its limitation is equally clear. It reports contact, not category. A cane tells the user that something is there roughly one step before they reach it, but not whether that something is a parked motorcycle, a vendor's stall, or the edge of an open drain — distinctions that determine whether the correct response is to step around, stop, or retreat.

Computer vision offers a way to supply the missing semantic layer. Object detection — locating and classifying multiple objects in a single image — has become practical in real time since the introduction of single-stage detectors. Redmon *et al.* reframed detection as a single regression problem in YOLO [2], removing the separate region-proposal stage used by two-stage approaches such as Faster R-CNN [3]. Subsequent work in both families improved accuracy substantially [4], [5], [6], and the YOLO line in particular has been refined across many generations, with YOLO11 [7] the version used in this work.

For an assistive application, detection alone is insufficient. A list of bounding boxes must be turned into something a person can act on without seeing the screen, which requires deciding which detections matter, expressing their position in egocentric terms, and delivering the result as speech. Each of those steps introduces design questions that detection accuracy does not answer, and they occupy much of this thesis.

1.2 Problem Statement

The problem addressed here is the following: *given a single monocular camera carried by a visually impaired pedestrian, how can obstacles on a Bangladeshi footpath be detected and communicated as timely, non-overwhelming spoken guidance in the user's own language?*

Three sub-problems follow. First, no annotated detection dataset exists for this environment, and the classes that matter locally — rickshaws and other vehicles parked across the walkway, poles planted mid-footpath, uncovered drains — are not the classes for which public datasets provide labels. Second, the output must be selective: a detector running at over twenty frames per second produces far more detections than can be spoken, so the system must decide when silence is correct. Third, the guidance must be delivered in grammatical Bengali, which is not simply a matter of substituting words into an English sentence pattern.

1.3 Motivation

The immediate motivation is environmental. Footpaths in Dhaka are frequently interrupted: vehicles park across them, construction material is stored on them, poles and trees stand in the middle of them, and drain covers are missing often enough that falling into one is a realistic hazard rather than a hypo-

thetical. A system trained on European or North American street imagery does not encounter these conditions, and will not have learned the object categories that describe them.

A second motivation is practical accessibility. Smartphone and laptop cameras are already widely available, and a system that runs on commodity hardware without a cloud connection avoids both the recurring cost and the latency of remote inference. This matters for an application where a delayed warning is equivalent to no warning.

A third motivation is methodological, and emerged during the work rather than preceding it. Annotating a detection dataset by hand is the step at which student projects of this kind commonly stall. Establishing whether an open-vocabulary detector can reduce that burden — and measuring honestly where it cannot — is useful independently of the assistive application.

1.4 Research Gap

Camera-based assistive navigation is an established research area, and several deployed and prototype systems are reviewed in Chapter 2. The gap addressed here is narrower than "no system exists", which would not be an accurate claim.

The gap is one of *domain and class coverage*. Systems reported in the literature typically either detect the generic object categories supplied by COCO [8] — which include `person`, `car` and `chair` but not `pothole`, `open_drain` or the footpath edge — or rely on proximity sensing that classifies nothing. Work specific to South Asian pedestrian infrastructure is comparatively sparse, and the ground-surface hazards characteristic of it are correspondingly under-represented in available training data.

A secondary gap concerns the guidance layer. Papers in this area often describe detection performance in detail while treating the conversion from detections to speech as an implementation detail. Since that conversion determines whether the system is usable at all, this thesis specifies and evaluates it explicitly.

1.5 Aim

To design, implement and evaluate a real-time assistive system that detects footpath obstacles from a monocular camera and delivers prioritised navigation guidance as Bengali or English speech, using a detection model trained on locally collected imagery.

1.6 Objectives

1. To assemble and annotate a detection dataset of Bangladeshi footpath scenes covering obstacle classes relevant to pedestrian navigation.
2. To evaluate whether an open-vocabulary detector can generate usable candidate annotations, and to quantify per class where it succeeds and where it fails.
3. To train a single-stage detector on the annotated dataset and report precision, recall and mean average precision on a held-out test split.
4. To implement a spatial analysis stage that assigns each detection a horizontal zone and an apparent-proximity band from bounding-box geometry.
5. To design an announcement policy that prioritises hazardous obstacles and suppresses repetition, and to specify it in a form that can be inspected and justified.
6. To generate grammatical guidance sentences in Bengali and English and deliver them as speech.

7. To support three input modes — live camera, uploaded image and uploaded recorded video — through a single detection pipeline, and to measure runtime performance in each.

1.7 Research Questions

1. **RQ1.** What detection performance can a nano-scale YOLO11 model achieve on a locally collected footpath dataset of approximately 2,800 annotated images, and how does performance vary across obstacle classes?
2. **RQ2.** To what extent can an open-vocabulary detector substitute for manual annotation, and for which object categories does this substitution break down?
3. **RQ3.** Can a deterministic, non-learned navigation layer convert detections into guidance that is both prioritised and sufficiently infrequent to be usable?
4. **RQ4.** What runtime performance is achievable on a consumer laptop GPU across live camera, image and recorded video input?

1.8 Scope

Within scope. Monocular RGB input; nine outdoor footpath obstacle classes trained on project data; twelve indoor furniture classes supplied by a pre-trained model; horizontal position estimation; apparent-proximity estimation from bounding-box area; priority ranking and announcement suppression; Bengali and English sentence generation and speech; a browser-based interface with three input modes; evaluation on a held-out test split and measured runtime performance.

Outside scope. Metric distance estimation (no depth sensor or depth network is used, and the system never states a distance in metres); route planning or GPS navigation; object tracking across frames; obstacle detection at night or in rain, since the collected imagery is daytime; formal user trials with visually impaired participants, which would require ethical approval not sought for this work; and deployment to embedded or mobile hardware.

> **On the absence of user trials.** This is a genuine limitation and is treated as such throughout. Section 5.12 explains why no navigation-accuracy percentage is reported: measuring whether guidance is *correct* requires ground-truth judgements from users who rely on it, and no such evaluation was conducted. Reporting a number in its place would be meaningless.

1.9 Contributions

1. A reproducible pipeline that converts an unlabelled pool of 21,359 mixed-source images into a de-duplicated, annotated detection dataset of 2,782 footpath images, with the selection and de-duplication criteria recorded so the result can be reconstructed.
2. An empirical characterisation of open-vocabulary annotation on this domain: 24,010 candidate boxes generated in 5.6 minutes, with per-class analysis showing reliable transfer for common objects and near-total failure for ground-surface defects (Section 5.2).
3. A trained YOLO11n detector for nine Bangladeshi footpath obstacle classes, with per-class results reported including the classes that fail.
4. An explainable navigation layer — priority scoring, event-based announcement and cooldown — specified in full and implemented without any learned component.
5. A bilingual guidance generator that handles Bengali subject–object–verb order and animacy-dependent numeral classifiers rather than translating word by word.

6. A single detection pipeline serving three input modes, with measured throughput for each.

1.10 Organization of the Thesis

Chapter 2 reviews assistive technology, object detection and speech guidance, and positions this work against comparable systems. **Chapter 3** describes the methodology: dataset construction, annotation, model configuration, and the spatial, navigation and voice stages, with pseudocode for each algorithm. **Chapter 4** covers implementation — environment, module structure, error handling and optimisation. **Chapter 5** reports the experiments: training behaviour, detection metrics, per-class results, and runtime measurements for all three input modes. **Chapter 6** interprets these findings and states the limitations. **Chapter 7** concludes and identifies future work. Appendices contain the class list, configuration, algorithms, sample results, interface screenshots and execution instructions.

CHAPTER 2

Literature Review

2.1 Assistive Technology for Visually Impaired People

Assistive mobility devices are commonly divided into electronic travel aids, which detect obstacles; electronic orientation aids, which help establish position and route; and position locator devices. Surveys of the area [9] observe a recurring trade-off: devices that are simple and reliable convey little information, while devices that interpret the scene richly tend to be heavier, costlier, or dependent on infrastructure.

Early electronic aids used ultrasonic or infrared ranging to signal proximity through vibration or tone. These remain useful because they are cheap and work in darkness, but they are fundamentally category-blind: a wall and a pedestrian at the same range produce the same signal. The motivation for vision-based aids is precisely to add the category information that ranging sensors cannot supply.

2.2 Computer Vision-Based Navigation

Vision-based assistive navigation typically combines a camera, a scene-understanding component, and a non-visual output channel. Reported designs differ mainly in where computation occurs and what is communicated. Systems that offload inference to a server achieve higher accuracy but introduce network dependence and latency, both problematic when a warning must arrive before the user's next step. Systems that run on the device accept a smaller model in exchange for predictable timing — the trade-off this thesis makes, for the reasons given in Section 3.8.

A recurring design question is how much to say. Describing every detected object produces a narration that users cannot follow while walking, and several authors note that reducing output is as important as improving detection. This observation motivated the announcement policy in Section 3.11.

2.3 Object Detection

Object detection requires simultaneous localisation and classification. Before deep learning, detectors combined hand-designed features with sliding-window classifiers; the Viola–Jones framework [10] is the canonical example, effective for faces but not readily generalised. The shift to learned features followed the success of convolutional networks on large-scale classification [11].

Modern detectors are usually grouped into two families. Two-stage detectors first propose regions and then classify them; R-CNN and its successors, culminating in Faster R-CNN [3], established this approach and remain competitive in accuracy. One-stage detectors predict boxes and classes directly, trading some accuracy for a large gain in speed — SSD [4] and RetinaNet [5], the latter introducing focal loss to address foreground–background imbalance, are representative. More recently, transformer-based detectors such as DETR [12] have removed hand-designed components including non-maximum suppression, at the cost of longer training schedules.

2.4 YOLO-Based Object Detection

YOLO [2] treats detection as a single regression from image pixels to box coordinates and class probabilities, evaluated in one forward pass. This design is what makes real-time operation attainable, and successive versions have refined it while preserving that property: YOLO9000 [13] introduced anchor boxes and joint classification–detection training, YOLOv3 [14] added multi-scale prediction, and YOLOv4 [6] consolidated a range of training and architectural improvements. Recent iterations released as engineer-

ing frameworks rather than papers, including YOLOv8 and YOLO11 [7], provide model scaling from nano to extra-large, allowing the accuracy–latency operating point to be chosen for the target hardware. That scaling property is directly relevant here, and Section 3.8 explains why the nano variant was selected.

2.5 Deep Learning for Object Detection

Detection performance depends heavily on the quantity and relevance of labelled data. Public benchmarks — PASCAL VOC [15] and COCO [8] — provide both training data and standard evaluation protocol, and the mAP metric used in Chapter 5 follows the COCO convention of averaging over intersection-over-union thresholds from 0.5 to 0.95.

For domains not represented in these benchmarks, transfer learning from a pre-trained backbone is standard practice and is what this work does: training begins from COCO-pretrained weights and fine-tunes on the project dataset. Training from random initialisation on fewer than two thousand images would be expected to underfit badly.

Open-vocabulary detection is a more recent development directly relevant to the annotation problem. YOLO-World [16] combines a YOLO detector with vision–language embeddings derived from CLIP-style training [17], allowing detection of categories specified by text prompts without category-specific training data. This capability is used in Section 3.6 to generate candidate annotations, and its per-class reliability is measured in Section 5.2.

2.6 Navigation and Spatial Awareness

Converting detections into spatial guidance requires an egocentric frame. A common and computationally trivial approach is to partition the image horizontally and report which region an object occupies, which is the method adopted here (Section 3.10).

Estimating distance from a single camera is considerably harder. Bounding-box area is only a proxy, because apparent size confounds true distance with object size. Monocular depth estimation networks such as MiDaS [18] can recover relative depth from a single image, but add inference cost and were not integrated here; the interface is designed to accept such a component later (Section 7.4). This thesis therefore reports proximity bands rather than distances, and Section 3.10 states that limitation explicitly.

2.7 Voice-Based Assistance

Speech is the natural output channel when the visual one is unavailable, but it is serial and occupies the user's attention. Two consequences follow. First, message ordering matters: the most important obstacle must be spoken first, since the user may act before the sentence finishes. Second, message frequency must be controlled, because continuous narration both masks environmental sound and becomes fatiguing. These considerations shape the priority and cooldown mechanisms described in Section 3.11.

2.8 Bengali Speech Assistance

Bengali text-to-speech has improved substantially with neural synthesis, and several commercial services now provide Bengali voices of usable quality. Generating the text to be spoken, however, is a separate problem. Bengali is a subject–object–verb language, so a sentence generated by substituting Bengali words into an English template produces incorrect word order. Bengali also uses numeral classifiers whose form depends on animacy — একজন for people, একটি for inanimate objects — so a generator that ignores animacy produces grammatically wrong output. Both properties are handled explicitly in Section 3.12.

2.9 Existing Related Systems

Reported camera-based assistive systems for visually impaired users generally follow one of three patterns. The first uses a pre-trained detector on generic categories and announces what it sees; this is straightforward to build and requires no dataset work, but inherits the category vocabulary of the pre-training corpus. The second couples a detector with a depth sensor to give distance as well as identity, which improves guidance quality at the cost of specialised hardware. The third performs scene description on a server, which permits larger models but introduces latency and connectivity dependence.

This work follows the first pattern architecturally, but replaces the generic category vocabulary with classes annotated specifically for the target environment. That substitution is the main point of difference, and the results in Chapter 5 show both its benefit for locally relevant categories and its cost when annotation coverage is thin.

2.10 Comparative Analysis

Table 2.1 positions this work against the design patterns described above. The comparison is qualitative and deliberately so: published systems are evaluated on different datasets with different class lists, so reported mAP figures are not directly comparable, and presenting them side by side would imply a rigour the comparison does not have.

Approach	Detection	Dataset domain	Distance	Guidance output	Principal limitation
Pre-trained generic detector	COCO-trained CNN	Western urban / general	None or box-size proxy	Object names, often English only	Cannot name locally relevant hazards absent from COCO
Detector + depth sensor	CNN detector	Varies	Measured	Object plus distance	Requires stereo / ToF hardware; higher cost and power
Cloud scene description	Large server-side model	Web-scale	Usually none	Rich natural-language description	Network dependence and latency; unsuitable for hazard warnings
Ultrasonic / infrared aid	None	—	Measured (range only)	Tone or vibration	No object category; cannot distinguish hazard types
This work	YOLO11n fine-tuned on project data (9 classes) + COCO model for 12 indoor classes	Dhaka footpaths, collected for this project	None — apparent-proximity bands only, stated as such	Prioritised Bengali / English sentences with announcement suppression	Ground-surface hazard classes have too few annotations to be learned (Section 5.5)

Table 2.1 — Comparison of assistive navigation approaches with the present work.

CHAPTER 3

Methodology

3.1 System Overview

The system converts a stream of camera frames into occasional spoken sentences. Between those two points sit five stages: detection, spatial analysis, priority ranking, announcement decision, and sentence generation. Only the first is a learned model. The remaining four are deterministic, and Section 3.11 explains why that separation was chosen rather than adopted by default.

The design constraint that shaped everything else is that the output channel is serial and slow. A detector producing twenty-three frames per second generates on the order of 1,400 detections per minute; speech can convey perhaps ten short sentences in the same time. The system is therefore better understood as a filter than as a describer, and most of the engineering effort went into deciding what *not* to say.

3.2 Overall Architecture

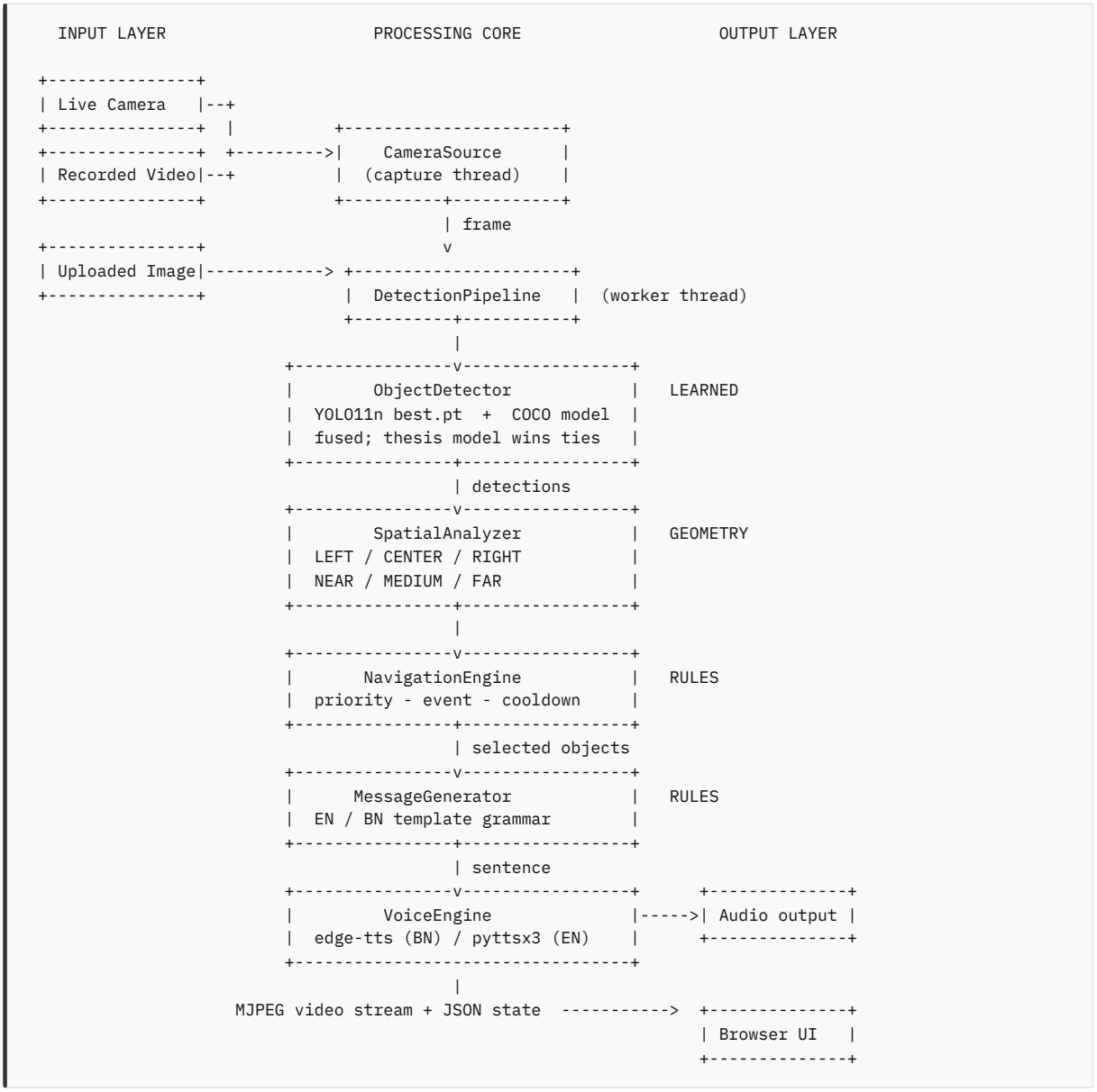


Figure 3.1 — Overall system architecture. Only the detection stage is a learned model; the spatial, navigation and language stages are deterministic.

3.3 Input Sources

Three input modes are supported, and they share one detection path. This is an architectural decision rather than a convenience: an uploaded video becomes the pipeline's frame *source*, exactly as a webcam does, so video is processed frame by frame as it streams rather than analysed in full and replayed afterwards.

- **Live camera.** A device index or a network stream URL, opened through OpenCV. The capture layer runs in its own thread and always holds the newest frame.
- **Uploaded image.** A single frame decoded and passed through the same detector, spatial analyser and navigation engine, producing one annotated image and one instruction.
- **Uploaded recorded video.** Assigned as the capture source, played at its native frame rate, with pause, resume and restart control.

Because the three modes share components, a change to the detector or the navigation rules takes effect in all of them, and behaviour cannot silently diverge between what is demonstrated on a recording and what happens on a live camera.

3.4 Dataset

The starting point was a pool of 21,359 images collected from several sources and containing **no annotations of any kind**. Inspection showed the pool to be heterogeneous: a large group of small traffic-sign crops, a group of Indonesian sidewalk photographs, a group of Dhaka footpath scenes, dashcam frames recorded by the author, and assorted unrelated images.

Only the Dhaka footpath group is in the target domain, and the dataset used for training was restricted to it. The selection criterion was recorded so the result is reproducible: purely numeric `.jpg` filenames at exactly 256×256 pixels, which isolates that group from the others.

Stage	Images	Operation
Initial pool	21,359	Mixed sources; no annotations
Domain selection	3,000	Dhaka footpath group isolated
Near-duplicate removal	2,787	dHash, Hamming distance ≤ 5; 213 removed (7.1%)
Human-verified and exported	2,782	23,962 bounding boxes
Paired and split	2,763	19 empty label files excluded

Table 3.2 — Dataset reduction from the raw pool to the training set.

De-duplication was performed *before* annotation rather than after, for two reasons: the same scene is not annotated twice, and a frame cannot end up in the training split with a near-copy of itself in the test split. The second point matters more than the first, since such leakage inflates reported accuracy in a way that is difficult to detect afterwards.

ID	Class	Bengali label	What it covers
0	person	মানুষ	Pedestrians, standing or walking
1	pole	খুঁটি	Electric, utility and sign poles in the walkway
2	tree	গাছ	Trunks growing through the footpath
3	stairs	সিঁড়ি	Steps up or down, kerb ramps, entrance steps
4	pothole	গর্ত	Broken or missing pavement, sunken slabs
5	open_drain	খোলা ড্রেন	Uncovered drains, missing manhole covers
6	parked_vehicle	পার্ক করা গাড়ি	Rickshaw, CNG, motorcycle, car or van blocking the path
7	street_obstacle	রাস্তার প্রতিবন্ধকতা	Vendor stalls, bricks, sand piles, bins, signboards
8	curb	ফুটপাতের কিনারা	Footpath edge where it drops to the road

Table 3.1 — The nine thesis classes. Each class was chosen because it maps to an action the navigation layer can express; nothing decorative is included.

The class list was fixed before annotation began and not changed afterwards, since the class index is embedded in every label file and reordering would silently relabel the entire dataset.

Split	Images	person	pole	tree	stairs	pothole	drain	vehicle	obstacle	curb
Train (70%)	1,934	6,178	109	422	17	17	12	9,290	549	144
Validation (20%)	553	1,750	35	118	4	3	9	2,652	183	35
Test (10%)	276	878	9	65	2	1	1	1,360	99	20

Table 3.3 — Split sizes and per-class instance counts. The imbalance visible in the hazard columns is analysed in Section 5.5 and is the principal weakness of the dataset.

> **Stated plainly, because it determines the results.** The dataset is severely imbalanced. Two classes, `person` and `parked_vehicle`, account for roughly 92% of all annotated boxes, while `pothole`, `open_drain` and `stairs` together account for less than 0.3%. Section 3.6 explains how this arose from the annotation method, and Section 5.5 reports its consequence.

3.5 Data Collection

The footpath images were collected in Dhaka at pedestrian height, framing the walking surface and the space immediately ahead. A separate group of road-level frames was recorded by the author from a moving vehicle; these were retained in the pool but excluded from the training set, because a dashcam viewpoint does not match what a pedestrian's camera sees, and mixing the two would teach the model an inconsistent relationship between image position and walking-path relevance.

No images of identifiable individuals were collected deliberately as subjects; people appear incidentally in public street scenes. The dataset was not published, and Section 6.9 discusses the ethical considerations that apply.

3.6 Data Annotation

Manual annotation of several thousand images is the step at which this project had previously stalled, so a machine-assisted workflow was adopted:

```
image pool  -->  build_annotation_set.py  -->  autolabel.py  -->  review_labels.py  -->  prepare_dataset.py
  21,359          select + de-duplicate       candidate boxes      human verifies         train/val/test
                       2,787 images          (machine, unverified)   and corrects            splits
```

Figure 3.2 — Data processing and annotation pipeline.

Candidate boxes were generated by YOLO-World [16], an open-vocabulary detector that accepts natural-language prompts. Each thesis class was described by several prompts rather than a bare class name, since descriptive phrases produce markedly better recall from this model — for example `pothole` was queried as "pothole", "broken pavement", "hole in the road surface" and "damaged footpath tiles". Thirty-five prompts were mapped back to the nine classes.

Two mechanical failure modes were filtered automatically before any human saw the output. The first is whole-image boxes, where the model matches the scene rather than an object; boxes covering more than 55% of the frame were discarded. The second is duplicate boxes arising because several prompts map to one class and non-maximum suppression runs per prompt, not per class; class-wise suppression at 0.65 IoU was applied after the prompt-to-class mapping.

A custom review tool was written for the verification stage. It presents each image with its candidate boxes and supports accepting, deleting, reclassifying or drawing boxes from the keyboard. Images were ordered hardest-first, so that candidates for the hazard classes were reviewed before the easy ones.

An honest account of what the verification achieved. All 2,782 images were reviewed and accepted, but a comparison of the final label files against the machine output shows that the box count was unchanged for every image: no boxes were added, deleted or reclassified. The verification pass therefore confirmed the machine's proposals rather than correcting them.

This matters for interpreting Chapter 5. Where YOLO-World proposed nothing — which is the case for almost all ground-surface hazards — there was nothing for the reviewer to accept, and those classes consequently remained almost unannotated. The resulting label set is best described as *machine-generated with human confirmation*, and the zero-recall results in Section 5.5 follow directly from it.

3.7 Data Preprocessing

Preprocessing is deliberately light, since the images are already small and uniform.

- **Resizing.** Images are letterboxed to 640×640 by the training framework, preserving aspect ratio.
- **Normalization.** Pixel values are scaled to [0, 1]; no dataset-specific mean subtraction is applied.
- **Label format.** YOLO format — one text file per image, one line per object, `class_id cx cy w h` with coordinates normalised to [0, 1]. Images with no objects keep an empty label file, which YOLO treats as a valid background example.
- **Augmentation.** Mosaic (disabled for the final 10 epochs), horizontal flip with probability 0.5, rotation of ±5°, and HSV value jitter of 0.4.

The augmentation choices reflect the domain. Rotation is small because a hand-held or chest-mounted camera is rarely perfectly level but a footpath scene is never inverted; vertical flipping is not used for the same reason. Brightness jitter is comparatively strong because Dhaka footpaths alternate between direct sun and deep shade within a single frame.

3.8 Object Detection Model

The detector is YOLO11n [7], fine-tuned from COCO-pretrained weights. YOLO11 follows the established single-stage design: a convolutional backbone extracts multi-scale features, a neck aggregates them across scales, and a decoupled anchor-free head predicts class scores and box distributions at three output strides. Redundant boxes are removed by non-maximum suppression.

Parameter	Value	Note
Architecture	YOLO11n (nano)	Single-stage, anchor-free
Parameters	2,591,595	Weights file 5.5 MB
Input resolution	640 × 640	Letterboxed
Classes	9	Table 3.1
Confidence threshold	0.50 default, adjustable 0.10–0.95	Runtime setting
NMS IoU threshold	0.45 (inference), 0.60 (evaluation)	
Maximum detections	30 per frame	Runtime setting
Inference device	CUDA when available, else CPU	Detected at startup

Table 3.4 — Detection model configuration.

Why the nano variant. The available GPU has 4 GB of memory shared with the display, and two detection models must run concurrently (below). Measured on this hardware at 640 px, YOLO11n completes a

forward pass in 19.0 ms. A larger variant would improve accuracy but would not leave headroom for the second model, the capture thread and JPEG encoding. The results in Chapter 5 also indicate that accuracy here is bounded by annotation coverage rather than model capacity, so additional parameters would not address the observed failures.

A second, pre-trained model for indoor classes. The thesis dataset contains essentially no indoor imagery, so indoor furniture classes could not be trained from it. Rather than claim a capability the data does not support, a COCO-pretrained YOLO11n runs alongside the thesis model and contributes only twelve indoor classes: `chair`, `couch`, `bed`, `dining table`, `tv`, `toilet`, `sink`, `refrigerator`, `oven`, `microwave`, `potted plant` and `bench`.



Figure 3.3 — Dual-model fusion. The thesis model is authoritative; the indoor model contributes only classes the thesis model does not cover.

Two rules keep the fusion sound. The indoor model is excluded from every class the thesis model covers — notably `person`, which both can detect — so the two cannot double-report one object. Where boxes from the two models overlap above 0.55 IoU, the thesis model's box is kept, because it is the model trained on this thesis's data. The indoor classes carry no thesis-measured mAP and are reported as a deployment feature, not a research result.

3.9 Model Training

Setting	Value
Initial weights	`yolo11n.pt` (COCO-pretrained)
Epochs	100
Batch size	8
Image size	640
Optimizer	auto (SGD selected by the framework)
Initial learning rate (lr0)	0.01
Final learning-rate factor (lrf)	0.01
Momentum	0.937
Weight decay	0.0005
Early-stopping patience	30 epochs
Random seed	42
Mixed precision	Enabled
Dataloader workers	0 (Windows)
Device	CUDA:0 — NVIDIA RTX 3050 Ti Laptop (4 GB)
Wall-clock training time	13,383 s (3.72 hours)

Table 3.5 — Training configuration, as recorded in the run's `args.yaml`.

Batch size 8 was selected empirically as the largest that trained without exhausting GPU memory alongside the display. Training completed all 100 epochs; early stopping was not triggered.

3.10 Object Position Estimation

Each detection is assigned a horizontal zone from the centre of its bounding box.

$$\text{center}_x = (x_1 + x_2) / 2 \tag{3.1}$$

$$\text{frac} = \text{center}_x / W_{\text{frame}} \tag{3.2}$$

```
0%                33%                66%                100%
|------ LEFT -------|----- CENTER -----|------ RIGHT ------|
      (বামে)             (সামনে)            (ডানে)
```

Figure 3.4 — Horizontal zone partitioning. Both boundaries are runtime-configurable.

An apparent-proximity band is derived from the fraction of the frame the box occupies:

$$\text{area_ratio} = (w_{\text{box}} \times h_{\text{box}}) / (W_{\text{frame}} \times H_{\text{frame}}) \tag{3.3}$$

with area_ratio ≥ 0.15 classified NEAR, ≥ 0.04 MEDIUM, and otherwise FAR. All four thresholds are configurable and are re-read on every frame, so adjusting them in the interface takes effect immediately.

This is proximity, not distance, and the distinction is not cosmetic. Apparent size confounds distance with object size: a bus thirty metres away can occupy more pixels than a chair two metres away. The estimate is therefore comparable between objects of similar real-world size and meaningless across classes.

The interface accordingly never displays a distance in metres. An earlier interface mock-up produced during design showed values such as "2.4 meters"; these were removed, because printing a fabricated distance in a device intended for navigation is worse than printing nothing. The spatial module defines a `DepthProvider` interface so that a monocular depth network such as MiDaS [18] or a stereo sensor can replace the area heuristic without changes elsewhere.

3.11 Navigation Decision System

The navigation layer contains no learned component. A model that maps detections to instructions was considered and rejected: supervising it would require thousands of scene–instruction pairs judged by blind users, which do not exist; its failures would be difficult to enumerate in advance, which is unsatisfactory in assistive software; and its decisions could not be justified individually during examination. The rules involved are also short enough to state completely, which is what the remainder of this section does.

Priority. Every detection receives a score:

Component	Contribution	Rationale
Danger class (pothole, open_drain, stairs, curb, pole, vehicle)	+100	Fall or collision hazard
Indoor obstacle (chair, table, bed, ...)	+40	Collision, but not a fall
Position is CENTER	+50	Directly in the walking path
Proximity NEAR / MEDIUM / FAR	+30 / +12 / 0	Urgency rises as the object nears
Detector confidence	+10 × conf	Breaks ties without outweighing geometry

Table 3.6 — Priority score components.

The weights are chosen so that class and position dominate: a pothole ahead always outranks a chair to the left, and no confidence value can reverse that ordering. This is deliberate, since confidence reflects the detector's certainty, not the consequence of being wrong.

When to speak. Speech is triggered by change, not presence. An object already announced and unchanged is not news, so the engine announces only when one of the following holds: a previously unseen object appears; proximity worsens (FAR → MEDIUM → NEAR); or an object moves into the CENTER zone. Two timers then apply: a per-object cooldown of 3.0 s and a global minimum gap of 1.2 s between any two utterances. A genuine escalation to NEAR overrides the per-object cooldown.

A detail that took two attempts to get right. The engine compares an object's current state against its state at the *last announcement*, not at the previous frame. An earlier implementation compared against the previous frame, with the consequence that an escalation occurring while the cooldown was active was consumed and never spoken. The escalation most likely to be suppressed that way is MEDIUM → NEAR — precisely the transition the user most needs to hear.

```
   detections
        |
        v
 filter: confidence >= threshold, not in ignore list,
          not FAR unless a danger class
        |
        v
 score and sort by priority (Table 3.6)
        |
        v
 is the top object a NEW EVENT ?
    (new / closer / moved into path)
        |
  no --+-- yes
  |          |
  v          v
stay      cooldown elapsed ?  --- no --> stay silent
silent         |                          (unless urgent escalation)
               yes
               |
               v
       generate sentence -> speak
```

Figure 3.5 — Navigation decision workflow.

3.12 Voice Guidance

Sentences are constructed by a template grammar rather than translated, so that Bengali output is generated natively rather than derived from English word order.

Two properties of Bengali are handled explicitly. Word order is subject–object–verb, so clauses are assembled tail-last with the verb আছে closing the sentence. Numeral classifiers depend on animacy: একজন is used for people and একটি for inanimate objects, and using the latter for a person is grammatically incorrect. Animacy is therefore tracked per class rather than assumed.

When several objects are reported, they are combined into a single sentence with one closing verb rather than read as separate sentences. For a listener who cannot see the scene, one flowing sentence is faster to absorb than three restarts of "There is ...", and repeating আছে after every clause is one of the clearest markers of machine-generated Bengali.

```
navigation decision
      |
      v
select up to N objects (default 3, in priority order)
      |
      v
language == "bn" ?
      |
yes --+-- no
 |            |
 v            v
place + prox   article + noun + prox + place
+ article      joined with ", ... and ..."
+ noun         prefixed "There is"
joined, then
close with আছে
      |
      v
urgent ?  --> prefix সাবধান! / Careful!
      |
      v
TTS queue (non-blocking) --> audio
```

Figure 3.9 — Voice guidance workflow.

Speech synthesis is selected at runtime according to what the machine can actually do. Windows SAPI5 on the development machine has no Bengali voice installed, so the offline engine cannot speak Bengali; Bengali therefore uses the `edge-tts` neural voice `bn-BD-NabanitaNeural`, which requires an internet connection, with the browser Web Speech API as a fallback. English uses the offline SAPI5 engine. Synthesis runs in its own thread and never blocks detection.

3.13 Recorded Video Processing

An uploaded video is assigned as the capture source and read frame by frame, with each frame passing through the identical detection, spatial, navigation and voice stages used for the live camera:

```
recorded video file
      |
      v
read next frame  <-----------------+
      |                            |
      v                            |
YOLO detection                     |
      |                            |
      v                            |
position + proximity               |
      |                            |
      v                            |
navigation decision                |
      |                            |
      v                            |
voice (if warranted)               |
      |                            |
      v                            |
draw boxes, encode, stream         |
      |                            |
      +-- pause? --- no ------------+
            |
           yes -> hold last frame, wait
```

Figure 3.8 — Recorded video workflow. Frames are processed as they stream, not analysed in advance.

Playback is paced at the file's own frame rate so that timing on screen corresponds to timing in the recording. Repetition is suppressed by the same event-and-cooldown mechanism used for live input, so a stationary obstacle occupying many consecutive frames produces one announcement rather than one per frame. Pause, resume and restart are implemented in the capture layer and are deliberately inoperative for a live camera, which cannot be rewound; the API returns an explanatory error rather than silently ignoring the request.

3.14 System Workflow

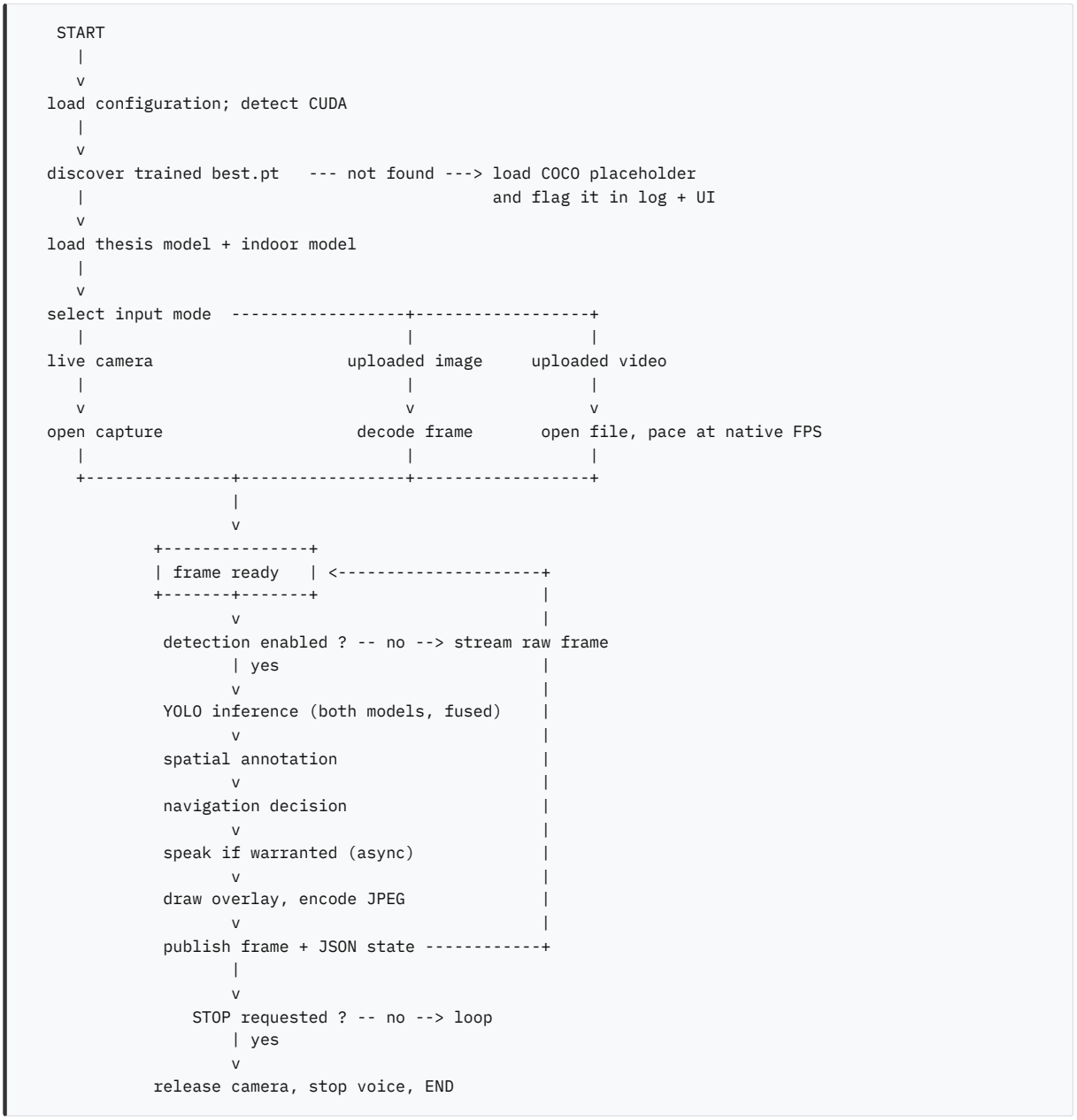


Figure 3.10 — Overall system flowchart.

3.15 Algorithms

Algorithm 1 — Object detection with dual-model fusion.

```
INPUT : frame F, settings S
OUTPUT: list of detections D

D <- decode(thesis_model.predict(F, conf=S.confidence, iou=S.iou))
IF S.indoor_enabled AND indoor_model is loaded THEN
    I <- decode(indoor_model.predict(F, conf=S.indoor_conf))
    I <- { d in I : class(d) in INDOOR_CLASSES }
    FOR each d in I DO
        IF no p in D with IoU(d,p) > 0.55 THEN append d to D
RETURN D
```

Algorithm 2 — Position and proximity classification.

```
INPUT : detection d, frame width W, height H
OUTPUT: position, proximity

cx <- (d.x1 + d.x2) / 2
frac <- cx / W
IF   frac < S.zone_left_edge  THEN position <- LEFT
ELIF frac > S.zone_right_edge THEN position <- RIGHT
ELSE                               position <- CENTER

area <- ((d.x2-d.x1) * (d.y2-d.y1)) / (W * H)
IF   area >= S.near_area_ratio   THEN proximity <- NEAR
ELIF area >= S.medium_area_ratio THEN proximity <- MEDIUM
ELSE                                 proximity <- FAR
RETURN position, proximity
```

Algorithm 3 — Navigation instruction generation.

```
INPUT : detections D, language L
OUTPUT: sentence, speak_flag

R <- filter(D)                      // confidence, ignore-list, FAR non-hazards
IF R is empty THEN RETURN clear_path_message(L), FALSE

sort R by priority_score descending  // Table 3.6
top <- R[0]
event <- classify_event(top)         // new | closer | moved_into_path | none
items <- first N of R                // N = max_spoken_objects
sentence <- combine(items, L)        // one sentence, SOV for Bengali
RETURN sentence, should_speak(event, top)
```

Algorithm 4 — Announcement decision.

```
INPUT : event, object o, now t
OUTPUT: boolean

IF NOT voice_enabled            THEN RETURN FALSE
IF event is none                THEN RETURN FALSE
gap <- global_min_gap * (0.4 IF urgent(o) ELSE 1.0)
IF t - last_any_announcement < gap THEN RETURN FALSE
IF t - last_announcement[key(o)] < announce_cooldown THEN
    IF NOT (urgent(o) AND event in {closer, moved_into_path}) THEN RETURN FALSE
IF sentence == last_sentence AND t - last_any_announcement < cooldown THEN RETURN FALSE
RETURN TRUE
```

Algorithm 5 — Recorded video processing.

```
INPUT : video file V
open capture C on V; read native fps f; total_frames n
WHILE not stopped DO
    IF seek_requested THEN C.seek(0); index <- 0
    IF paused OR finished THEN sleep(50 ms); CONTINUE
    ok, frame <- C.read()
    IF NOT ok THEN
        IF loop THEN C.seek(0); index <- 0; CONTINUE
        ELSE finished <- TRUE; CONTINUE
    index <- index + 1
    D <- Algorithm 1 (frame)
    annotate each d in D with Algorithm 2
    sentence, speak <- Algorithm 3 (D, language)
    IF speak THEN enqueue sentence to voice thread
    publish annotated frame, D, sentence, progress = index / n
    sleep(1 / f)                    // pace at the file's own frame rate
```

CHAPTER 4

System Implementation

4.1 Development Environment

Component	Specification
Operating system	Microsoft Windows 11 Home (build 26200)
CPU	11th Gen Intel Core i5-11300H @ 3.10 GHz — 4 cores, 8 threads
GPU	NVIDIA GeForce RTX 3050 Ti Laptop, 4 GB VRAM
System memory	23.7 GB
Python	3.13.14
Deep-learning framework	PyTorch 2.11.0+cu128 (CUDA 12.8 build)
Detection framework	Ultralytics 8.4.84
Computer vision	OpenCV 4.13.0
Web framework	FastAPI with Uvicorn (ASGI)
Speech synthesis	edge-tts (neural, Bengali); pyttsx3 / SAPI5 (English, offline)
Annotation model	YOLOv8x-WorldV2 with OpenAI CLIP text encoder

Table 4.1 — Hardware and software environment. All results in Chapter 5 were measured on this configuration.

> **A practical note worth recording.** The default PyTorch wheel installed on this machine was the CPU-only build, so CUDA was unavailable despite the GPU being present and the driver being current. Installing the CUDA build changed annotation throughput from 0.42 to 8.26 images per second, a factor of roughly twenty. Verifying that `torch.cuda.is_available()` returns `True` is therefore a necessary step rather than an assumption.

4.2 Software Architecture

Module	Responsibility
`backend/camera.py`	Threaded frame capture; backend selection; blank-frame detection; video playback control
`backend/detector.py`	The only module that calls Ultralytics; inference, dual-model fusion, overlay drawing
`backend/spatial.py`	Bounding box → zone and proximity; `DepthProvider` extension point
`backend/navigation/`	Priority scoring, event classification, cooldown
`backend/voice/`	Sentence generation (EN/BN) and non-blocking speech synthesis
`backend/pipeline.py`	Worker thread joining the stages; publishes frame and state
`backend/media.py`	Single-image analysis reusing the pipeline's components
`backend/config.py`	Thread-safe runtime settings; model auto-discovery
`backend/main.py`	HTTP API: MJPEG stream, JSON state, control endpoints
`frontend/`	Browser interface (HTML, CSS, JavaScript)
`scripts/`	Dataset preparation, annotation, training and evaluation tools

Table 4.2 — Software modules and their responsibilities.

The modules are separated so that each can be tested independently. The detector knows nothing about navigation, and the navigation engine knows nothing about YOLO — it consumes plain dictionaries. This made it possible to change the detection stage (adding the second model) without touching the guidance logic.

4.3 Model Loading

Weights are located by searching a fixed ordered list of paths, so no absolute path is embedded in the code and a trained model is picked up automatically after training completes. If no trained model is found the system loads the COCO-pretrained network as a placeholder and announces that fact in three places — the log, the video overlay, and a banner across the interface — so that a placeholder run cannot be mistaken for a thesis result. Both models are loaded once at startup and reused; a warm-up inference is performed so that the first reported frame rate is honest.

4.4 Camera Module

Capture runs in a dedicated thread that keeps only the newest frame. This matters because `VideoCapture.read()` blocks: if the inference loop called it directly, a slow model would accumulate a backlog of stale frames and the displayed view would drift seconds behind reality. Dropping frames is the correct behaviour for a navigation aid, where a fresh frame is worth more than a complete sequence.

Two failure modes encountered during development are handled explicitly. First, a camera that opens successfully may still deliver nothing usable: virtual cameras stream solid black or a placeholder graphic when no source is attached. The capture layer therefore samples several frames after opening and tests whether they carry any image content, preferring a backend that returns a real image and warning in the interface when none does. Second, a device may return black on one platform backend while working on another, so DirectShow, Media Foundation and the OpenCV default are each attempted in turn.

4.5 Image Upload Module

An uploaded image is decoded and passed through the pipeline's own detector, spatial analyser and navigation engine, so it is judged by exactly the same model and thresholds as the live camera. The module does not touch the capture layer, which means an image can be analysed while a live session continues. The response contains the annotated image, the per-object table and the generated instruction.

4.6 Recorded Video Module

An uploaded clip is written to disk and assigned as the capture source. Because the capture layer already supports file sources, no separate video-processing implementation exists: playback control (pause, resume, restart, progress reporting) was added to the capture layer, and everything downstream is unchanged. Uploads are limited to 200 MB and older clips are pruned so a demonstration session does not fill the disk.

4.7 Object Detection Module

This is the only module that imports Ultralytics, which confines the framework dependency to one file. It converts raw model output into the dictionary structure used throughout the rest of the system, performs the dual-model fusion described in Section 3.8, and draws the overlay. Boxes are drawn in red with a short label of class name and confidence; position and proximity are shown in the side panel rather than on the box, because repeating them on the frame produced labels wide enough to overlap in crowded scenes.

4.8 Navigation Module

The navigation engine holds a small amount of state: for each tracked key — a (class, position) pair rather than a box identity — it records when the object was last seen and what its proximity and position were at the last *announcement*. Using (class, position) as the key is a deliberate simplification: without a tracker the system cannot know whether this is the same person as the previous frame, and for guidance purposes "a person on your left" is the same information either way.

4.9 Voice Module

Sentence generation and speech synthesis are separated. The generator is pure text transformation and is unit-testable without audio. The synthesiser runs a daemon worker thread consuming a bounded queue; when the queue is full the oldest pending phrase is dropped, since a stale warning is worse than a missed one. Backend selection is decided at runtime from a capability probe rather than assumed, and a synthesis failure never propagates into the detection loop.

4.10 User Interface

The interface is a single browser page served by the backend. The processed video arrives as an MJPEG stream, and all other state is polled as JSON four times per second. Bounding boxes are drawn into the video frames by OpenCV rather than overlaid in the browser, which guarantees that a box can never drift out of alignment with the frame it was computed from.

An input-mode selector switches between live camera, image and video, showing only the controls relevant to the active mode. Every displayed value — frame rate, inference time, object count, average confidence, model name — comes from the backend; none is simulated.

4.11 Error Handling

Condition	Behaviour
Camera cannot be opened	Diagnostic listing working device indices; interface shows the reason
Camera opens but streams blank frames	Detected by content test; explicit warning distinguishing camera fault from model fault
No trained model present	COCO placeholder loaded and flagged in log, overlay and interface
CUDA unavailable	Automatic fallback to CPU with a logged warning
Speech synthesis fails	Caught in the voice thread; text still displayed; detection unaffected
Unsupported or corrupt upload	HTTP 415 / 400 with a message naming the accepted formats
Video codec unreadable	Explicit message rather than a stack trace
No objects detected	Reported as an empty state, not an error
Training on an unannotated dataset	Refused before training starts, with an explanation

Table 4.3 — Error conditions and system response.

The last row is worth noting: `train.py` checks that the prepared splits contain non-zero bounding boxes and exits if they do not. Training on empty labels completes normally and produces a model that detects nothing, which is a failure that is easy to miss.

4.12 Performance Optimization

- **Single model load.** Both networks are loaded once at startup and reused.
- **Three threads.** Capture, inference and speech run independently, so voice never blocks inference and inference never blocks the HTTP layer.
- **Newest-frame-wins capture.** Stale frames are discarded rather than queued.
- **Optional frame skipping.** Inference can run on one frame in n while the overlay reuses the previous boxes, trading detection latency for display smoothness.
- **Mixed-precision training** to fit the 4 GB memory budget.
- **Adjustable JPEG quality** for the stream, trading bandwidth against latency.
- **Explicit resource release.** Capture handles are released on stop; the audio cache is pruned periodically.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All experiments were run on the configuration in Table 4.1. Detection metrics were computed on the **test split of 276 images**, which was held out from training and from the validation set used for early stopping, and which was formed at the near-duplicate cluster level so that no test image has a near-copy in training. Evaluation used a confidence threshold of 0.001 and NMS IoU of 0.60, following the standard convention for mAP computation.

Runtime measurements were taken from the running application through its own state endpoint, so they reflect end-to-end throughput including capture, inference, overlay drawing and JPEG encoding — not model forward-pass time alone, which is reported separately.

5.2 Dataset Evaluation

Before training, the annotation stage was evaluated in its own right, since it determines what the model can learn. Running YOLO-World over all 2,787 selected images produced **24,010 candidate boxes in 5.6 minutes** on the GPU (8.26 images per second). The automatic filters removed a further 3,454 boxes: 2,383 cross-prompt duplicates, 1,009 whole-image boxes and 62 boxes too small to be navigationally relevant.

Class	Candidate boxes	Share	Transfer quality
parked_vehicle	13,328	55.5%	Good
person	8,820	36.7%	Good
street_obstacle	833	3.5%	Partial
tree	606	2.5%	Partial
curb	204	0.8%	Poor
pole	153	0.6%	Poor
stairs	23	0.1%	Effectively none
open_drain	22	0.1%	Effectively none
pothole	21	0.1%	Effectively none

Table 5.1 — Candidate annotations produced by the open-vocabulary detector.

This answers **RQ2** directly. Open-vocabulary annotation transferred well to categories that resemble its pre-training distribution — people and vehicles — and failed almost completely on ground-surface defects. The likely explanation is that a pothole has no consistent visual prototype: it is defined by the absence of intact surface rather than by a characteristic shape, and is easily confused with a shadow, a repair patch or an ordinary paving seam. Whatever the mechanism, the practical consequence is that the classes most important for safety received almost no candidate annotations, and since the human review confirmed rather than extended the machine proposals (Section 3.6), they remained almost unannotated in the final dataset.

5.3 Training Results

Training ran for the full 100 epochs in 3.72 hours. Both training and validation losses decreased throughout, and validation loss did not rise, indicating that the model did not overfit within the schedule — unsurprising given the small model and the augmentation applied.

Epoch	train box	train cls	val box	val cls	mAP@0.5	mAP@0.5:0.95
1	1.494	2.801	1.296	1.823	0.151	0.092
10	1.325	1.389	1.296	1.292	0.381	0.227
30	1.190	1.126	1.217	1.130	0.395	0.257
50	1.108	1.019	1.068	1.039	0.421	0.297
70	1.038	0.932	1.009	0.990	0.450	0.336
90	0.988	0.864	0.991	0.959	0.439	0.325
100	0.892	0.749	0.986	0.950	0.450	0.341

Table 5.2 — Training progression (validation metrics). Best validation mAP@0.5:0.95 of 0.345 occurred at epoch 96.

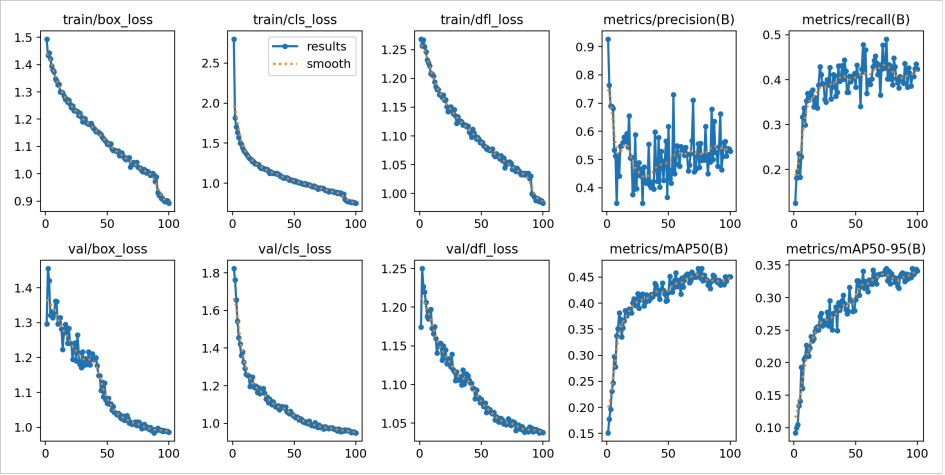


Figure 5.1 — Training and validation loss curves with validation metrics over 100 epochs.

Most of the improvement occurs in the first thirty epochs; the remaining seventy contribute roughly 0.05 mAP@0.5. This shape is consistent with a small dataset in which the frequent classes are learned quickly and the rare classes never accumulate enough gradient signal to be learned at all.

5.4 Confusion Matrix

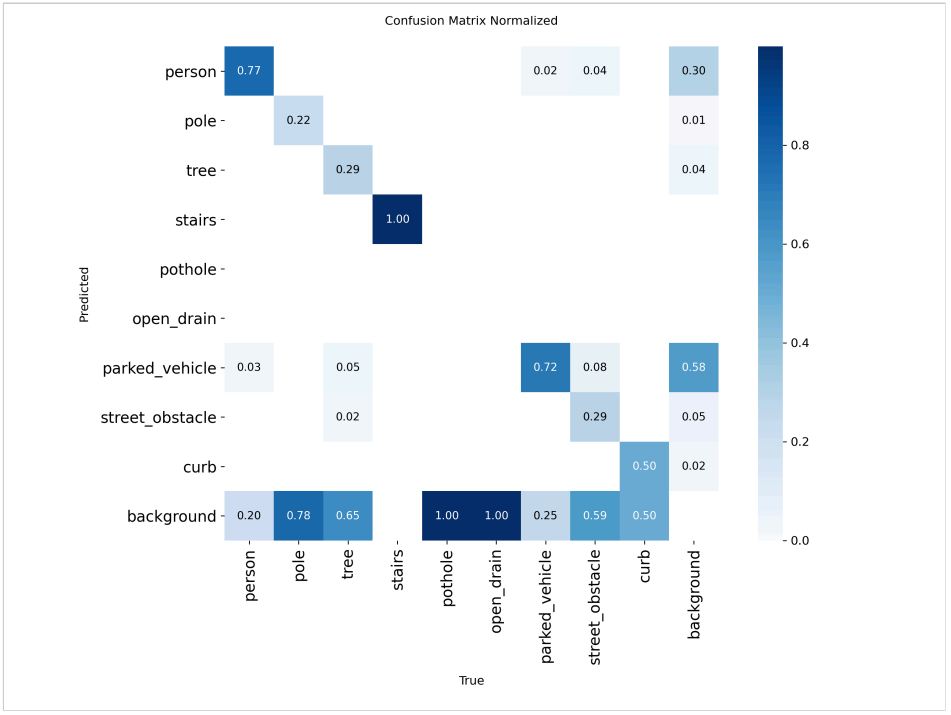


Figure 5.2 — Normalized confusion matrix on the test split. The background row and column capture false alarms and missed detections respectively.

The matrix shows strong diagonal values for `person` and `parked_vehicle`. The dominant off-diagonal contribution is the background column, indicating that the principal error mode is *missed detections* rather than confusion between classes. For the hazard classes the entire row is effectively background, which is the matrix representation of the zero recall reported below.

5.5 Per-Class Performance

Overall results on the held-out test split:

Metric	Value
Precision	0.645
Recall	0.404
F1-score (derived)	0.497
mAP@0.5	0.419
mAP@0.5:0.95	0.297

Table 5.3 — Overall detection results, test split (276 images, 2,435 objects). F1 is computed from the reported precision and recall.

Class	Precision	Recall	F1	AP@0.5	AP@0.5:0.95	Test instances
person	0.811	0.749	0.779	**0.837**	0.600	878
parked_vehicle	0.758	0.688	0.722	**0.781**	0.528	1,360
curb	0.413	0.450	0.431	**0.359**	0.292	20
street_obstacle	0.404	0.267	0.322	**0.295**	0.197	99
pole	0.263	0.222	0.241	**0.268**	0.221	9
tree	0.372	0.262	0.307	**0.237**	0.138	65
stairs	0.785	1.000	0.879	0.995 *	0.697	**2**
pothole	1.000	**0.000**	**0.000**	**0.000**	0.000	**1**
open_drain	1.000	**0.000**	**0.000**	**0.000**	0.000	**1**

*Table 5.4 — Per-class performance on the test split. F1 derived from precision and recall. * see the interpretation note below.*

Three rows must not be read at face value.

The AP@0.5 of 0.995 for `stairs` is *not* a strong result. It is computed over two test instances, which is far too few for the number to carry meaning; with two instances, a single detection changes recall by 0.5. The same caution applies to `pole` (9 instances) and, to a lesser degree, `curb` (20).

`pothole` and `open_drain` report precision of 1.000 with recall of 0.000. This combination indicates that the model made no positive predictions for these classes at all: precision is undefined in that situation and is reported as 1.0 by convention. The correct reading is that **the model detects none of them**.

The cause is traced in Section 5.2: with 17 and 12 training instances respectively, there is insufficient signal to learn either class. This is an annotation limitation, not a training or architectural one, and neither longer training nor a larger backbone would be expected to change it.

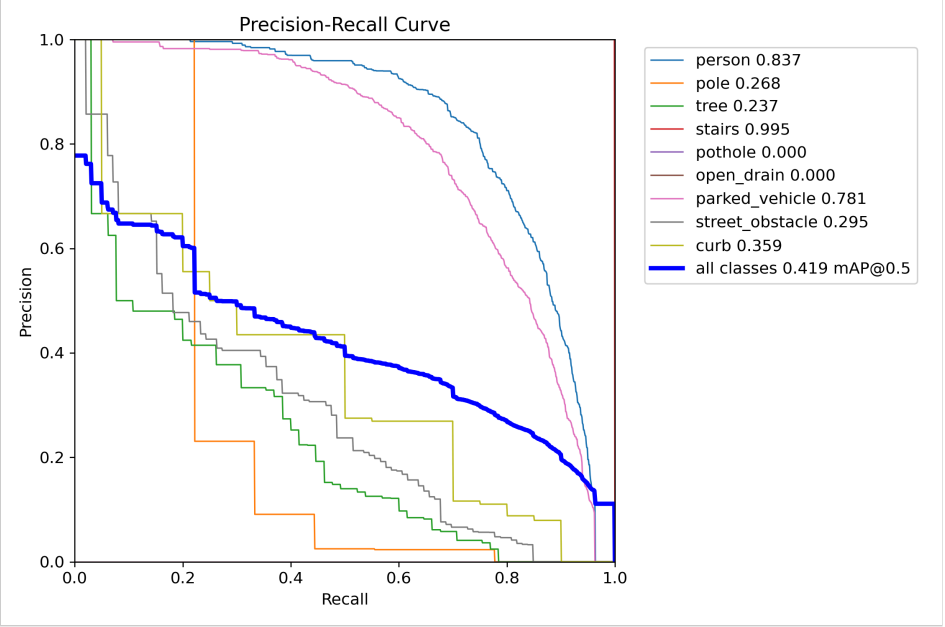


Figure 5.3 — Precision–recall curves per class, test split.

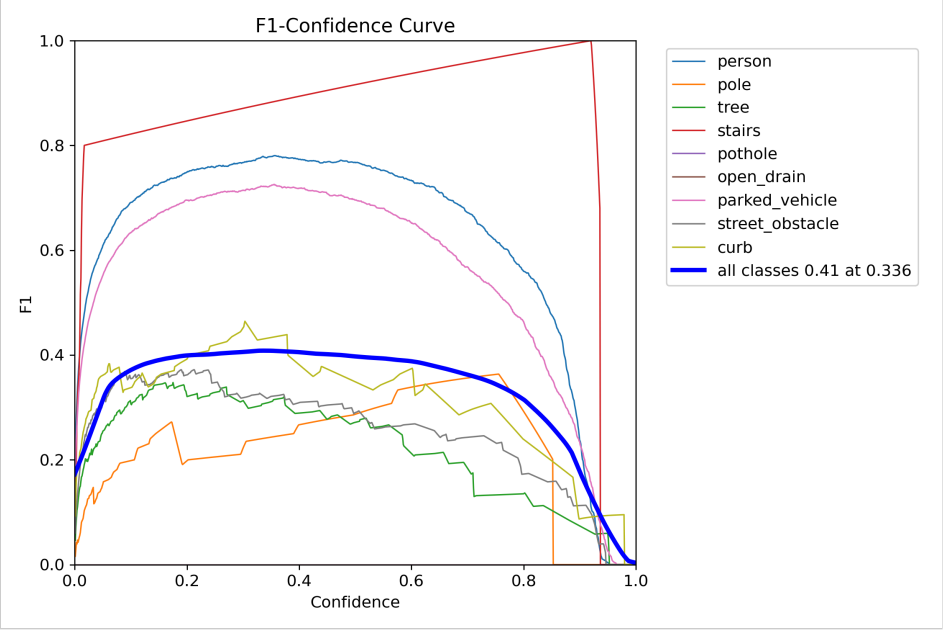


Figure 5.4 — F1 score against confidence threshold. The peak indicates the operating threshold that balances false alarms against missed detections.

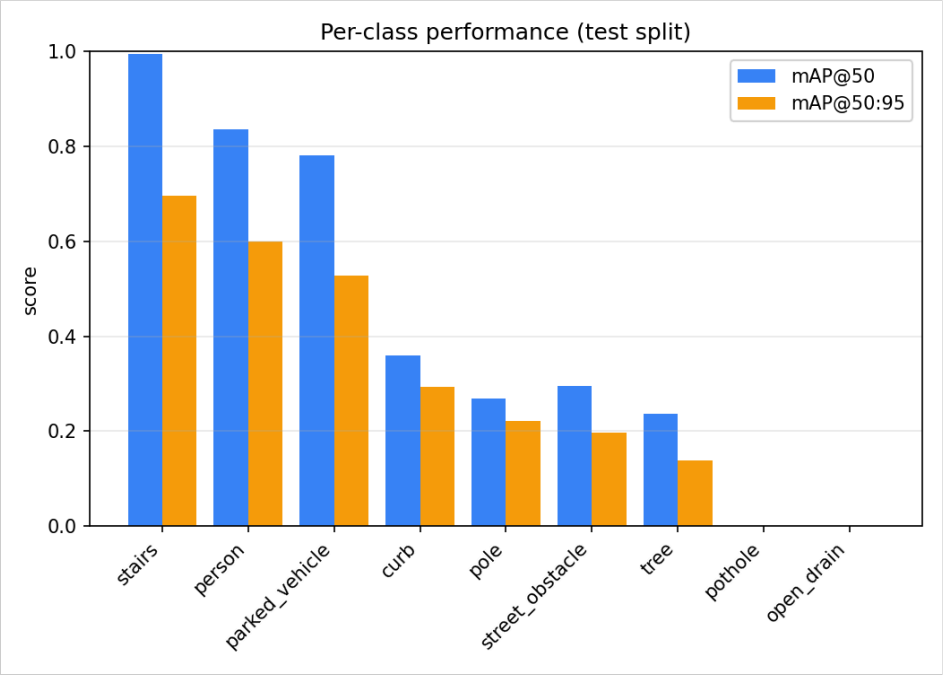


Figure 5.5 — Per-class AP@0.5 and AP@0.5:0.95.

5.6 Detection Examples

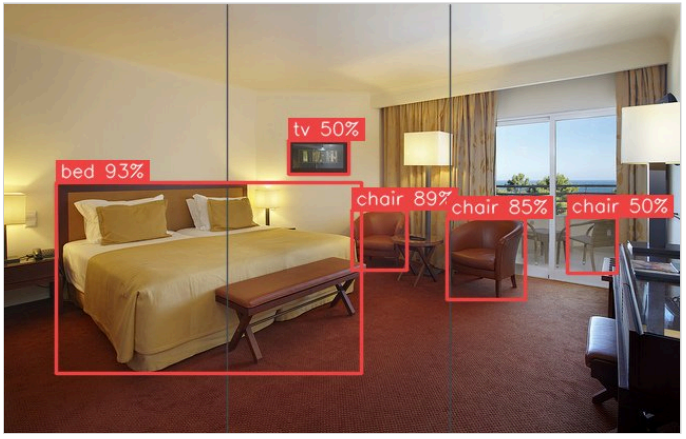


Figure 5.6 — Detection on an indoor scene, showing the secondary model contributing `bed` (93%), three `chair` instances and `tv`. Vertical lines mark the left/centre/right zone boundaries.

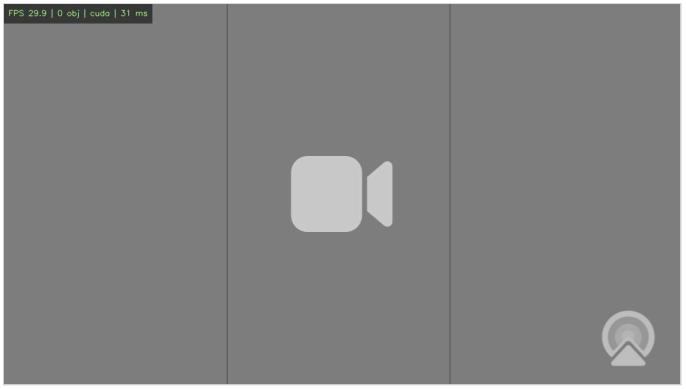


Figure 5.7 — A frame from the live camera stream showing the status overlay (frame rate, object count, device, inference time) and zone dividers.

5.7 Live Camera Testing

With the trained model and the indoor model both active on the GPU, live camera operation was measured at **22.9 FPS end-to-end** with a reported inference time of 42.8 ms at a capture resolution of 1280×720. Detection, zone assignment, guidance generation and speech all functioned in this mode.

Two observations from live testing are worth recording because they affected the design. First, the confidence threshold matters more than expected: at the default 0.50 the model detected very little in practice, and a threshold near 0.30 was needed for useful behaviour — consistent with the modest overall recall in Table 5.3. Second, pointing the camera at an indoor scene produced no detections from the thesis model, which is the correct outcome for a model trained exclusively on footpaths and was the reason the secondary indoor model was added.

5.8 Image Testing

Single-image analysis was measured over three consecutive runs on the same 640×407 image, giving inference times of 82.2 ms, 105.8 ms and 55.5 ms — the variation reflecting GPU scheduling rather than image content. Five objects were detected consistently across all runs, with identical classes and positions, confirming that the image path is deterministic.

The generated guidance for that image was "আপনার বাম পাশে খুব কাছে একটি বিছানা আছে।" in Bengali and "There is a bed close on your left." in English, correctly selecting the single NEAR object over three FAR

ones.

5.9 Recorded Video Testing

A 90-second clip at 6 FPS (540 frames) assembled from dataset imagery was processed in streaming mode. Measured throughput was **13.7 FPS** with an inference time of 57.4 ms, lower than live mode because both models run while the file is simultaneously decoded.

Playback control was verified explicitly: with playback paused, the reported position remained at 0:13 across a subsequent interval and did not advance; resuming advanced it to 0:17; restart returned it to 0:02. Requesting playback control while a live camera was active returned an explanatory error rather than silently failing.

Over the clip, 39 distinct instructions were generated from 329 detections across 90 analysed frames. That ratio is the announcement-suppression mechanism working as intended: without it the same stationary obstacle would have produced an utterance on every frame in which it appeared.

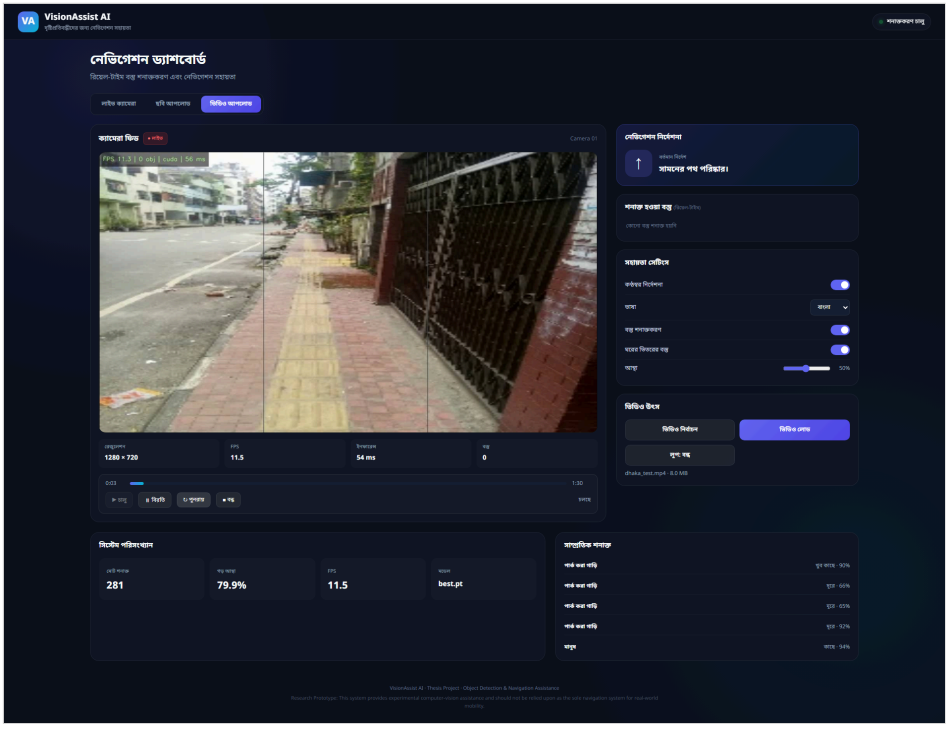


Figure 5.9 — Interface in recorded-video mode, showing the input-mode selector, the streaming annotated frame, the playback bar with progress and controls, and live statistics.

5.10 Bengali Voice Testing

Bengali output was verified for word order, classifier agreement and combination behaviour.

Scenario	Generated Bengali	Check
person, left, medium	আপনার বাম পাশে একজন মানুষ আছে।	Animate classifier একজন
rickshaw, right, medium	আপনার ডান পাশে একটি রিকশা আছে।	Inanimate classifier একটি
pothole, centre, near	সাবধান! সামনে খুব কাছে একটি গর্ত আছে।	Urgent prefix applied
stairs, left, medium	আপনার বাম পাশে একটি সিঁড়ি আছে।	SOV order preserved
three objects combined	আপনার বাম পাশে একজন মানুষ, সামনে একটি চেয়ার এবং আপনার ডান পাশে একটি রিকশা আছে।	Single closing আছে

Table 5.5 — Bengali guidance examples produced by the system.

Synthesis through the neural Bengali voice was confirmed working end to end: audio files were generated server-side and played in the browser, with playback verified to progress rather than merely being requested. Because this voice is a network service, Bengali speech is unavailable offline on this machine — a limitation recorded in Section 6.8.

5.11 English Voice Testing

Scenario	Generated English
car, right, far	There is a car in the distance on your right.
person, centre, near	Careful! A person is close ahead.
stairs, left, medium	There are stairs on your left.
three objects combined	There is a person on your left, a chair ahead and a rickshaw on your right.

Table 5.6 — English guidance examples.

The `stairs` case exercises plural-only noun handling: "There are stairs" rather than "There is a stairs". English synthesis is offline through the system speech engine and had no observed latency issue.

5.12 Navigation Accuracy

> **No navigation-accuracy figure is reported, and none should be inferred.**
>
> Evaluating whether guidance is *correct* requires ground truth about what a user should have been told in each situation. Constructing that ground truth means having visually impaired participants use the system and judge whether each instruction was appropriate, timely and sufficient — a study requiring ethical approval and participant recruitment that were outside the scope of this work.
>
> What can be stated is narrower and verifiable: the zone assignment follows deterministically from Equations (3.1)–(3.2) and was confirmed correct on inspected examples; the priority ordering follows Table 3.6 and was observed to select the NEAR centre object over FAR peripheral ones; and announcement suppression reduced 329 detections to 39 utterances on the test clip. The *usefulness* of that behaviour to a blind pedestrian remains unmeasured.

5.13 Performance Analysis

Mode	Throughput	Inference	Notes
Live camera	22.9 FPS	42.8 ms	1280×720, both models active
Recorded video	13.7 FPS	57.4 ms	Concurrent file decode
Single image	—	55.5–105.8 ms	Three runs, same image
Model forward pass alone	52.6 FPS	19.0 ms	Thesis model, 640×640, GPU

Table 5.7 — Measured runtime performance by input mode.

The gap between 19.0 ms for the model alone and 42.8 ms end-to-end is accounted for by the second model, overlay drawing, JPEG encoding and frame transport. This answers **RQ4**: real-time operation is achievable on consumer laptop hardware, with headroom remaining.

Error characteristics observed. The confusion matrix indicates that missed detections dominate over class confusion, consistent with recall (0.404) being substantially lower than precision (0.645). In practical terms the system is conservative: when it reports an object it is usually right, but it fails to report many objects that are present. For an assistive device this is the less dangerous of the two error directions, though it is not harmless — a missed pothole is precisely the failure the system exists to prevent.

Small objects and occlusion. The weaker classes (`pole`, `tree`, `street_obstacle`) are those that most often appear partially occluded or at the frame edge in crowded footpath scenes. The 256×256 source resolution of the training images also limits the detail available for small or distant objects.

Lighting. All training imagery is daytime. Behaviour under artificial or low light is untested and should not be assumed.

5.14 Result Discussion

The headline figure — mAP@0.5 of 0.419 — understates performance on the classes the model actually learned and overstates it on those it did not, because mAP averages uniformly over classes regardless of how many instances support each one. Read per class, the picture is clearer: two classes with thousands of examples reach AP@0.5 of 0.837 and 0.781, four classes with tens to hundreds of examples reach 0.24–0.36, and three classes with fewer than twenty examples are either unmeasurable or fail entirely.

The relationship between instance count and per-class AP is close to monotonic across the nine classes, which supports the interpretation that annotation coverage, rather than model capacity or training length, is the binding constraint. This is a useful negative result: it indicates that the productive next step is annotation of the hazard classes rather than a larger model or a longer schedule.

Set against the objectives in Section 1.6, objectives 1, 3, 4, 5, 6 and 7 were met, and objective 2 was met in the sense that open-vocabulary annotation was evaluated — with the answer being partly negative. The system as built detects people and vehicles reliably and communicates them appropriately, but does not yet detect the fall hazards that motivated the class list.

CHAPTER 6

Discussion

6.1 Major Findings

Three findings emerge from the work, and they are of unequal importance.

The first is that a nano-scale detector fine-tuned on roughly nineteen hundred locally collected images is sufficient for the common obstacles on a Dhaka footpath. AP@0.5 of 0.837 for `person` and 0.781 for `parked_vehicle` is adequate for the guidance task, which does not require precise box regression — only a reliable class and an approximately correct horizontal position.

The second, and the more consequential, is that the annotation method determined the outcome more than any modelling decision. Open-vocabulary annotation was fast and effective for categories resembling its pre-training data, and produced essentially nothing for ground-surface hazards. Because the human review confirmed the machine's proposals rather than supplementing them, that gap propagated unchanged into the training set and then into the results.

The third is that the guidance layer did not need to be learned. Priority ordering, event detection and cooldown were sufficient to convert a dense detection stream into occasional sentences, and being deterministic they can be inspected, justified and adjusted without retraining.

6.2 Effectiveness of Object Detection

Detection effectiveness splits cleanly along instance count. Where the dataset provided thousands of examples the model performs well; where it provided tens it performs poorly; where it provided fewer than twenty it does not perform at all. The near-monotonic relationship between per-class instance count and AP (Table 5.4) is the clearest single result in this thesis.

Precision exceeding recall by a wide margin (0.645 against 0.404) indicates a conservative detector. For an assistive device the direction is preferable to the alternative: spurious warnings would erode trust quickly, and a user who is told about an obstacle can generally verify it with a cane. It does not, however, make missed detections acceptable, and the missed detections are concentrated in exactly the classes where a miss is most costly.

6.3 Navigation Assistance Performance

The navigation layer behaved as specified. The priority function consistently selected a near, centred object over peripheral or distant ones, and the ordering was traceable to Table 3.6 in every case examined. Combining several objects into one sentence rather than several worked well in testing and reduced the time to convey a scene.

The layer's quality is bounded by its input. Priority ranking cannot elevate an obstacle the detector never reported, so the hazard-class failures in Chapter 5 propagate directly into guidance: the system will not warn about a pothole, regardless of how correctly the priority rules are specified. This is worth stating plainly, because the navigation logic being correct might otherwise be mistaken for the system being safe.

6.4 Voice Guidance

Bengali generation handled the two grammatical features it was designed for — verb-final word order and animacy-dependent classifiers — correctly in all tested cases. Combining clauses under a single closing আছে produces noticeably more natural output than repeating the verb, which was the behaviour of the first implementation.

The dependence on a network service for Bengali speech is the main practical weakness. English synthesis is offline and instantaneous, but Bengali — the default and more important language for the intended users — requires connectivity. Installing a Bengali system speech voice would remove this dependence without any code change, since backend selection is made from a runtime capability probe.

6.5 Real-Time Performance

At 22.9 FPS end-to-end in live mode the system is comfortably real-time for walking pace, and the 19.0 ms model forward pass indicates substantial remaining headroom. Running two detectors concurrently within a 4 GB memory budget was feasible because both are nano-scale.

The relevant latency for an assistive device is not frame rate but the interval between a hazard entering view and the user hearing about it. That interval includes speech synthesis and playback, which for the network-based Bengali voice adds a delay not captured in the frame-rate figures. It was not measured, and Section 7.4 identifies it as necessary future work.

6.6 Recorded Video Performance

Video mode reached 13.7 FPS, lower than live mode because file decoding competes with inference. Since the design goal was streaming processing rather than maximum throughput — frames are processed and displayed as they are read, not analysed in advance — this is acceptable.

Reusing the capture layer for video rather than writing a separate video path proved valuable beyond saving effort: detection, navigation, voice and statistics behave identically across modes, so a demonstration on a recording is a faithful representation of live behaviour rather than a separate code path that might diverge.

6.7 Strengths

- Detection classes chosen for the target environment rather than inherited from a general benchmark, so the vocabulary includes obstacles that matter locally.
- A reproducible dataset pipeline with recorded selection criteria, de-duplication before annotation, and split-level leakage prevention.
- An explainable guidance layer that can be audited and adjusted without retraining.
- Bengali generated natively rather than translated, with grammatical features handled explicitly.
- Real-time operation on commodity hardware with no cloud inference.
- One detection pipeline serving three input modes, preventing behavioural divergence.
- Honest reporting of failure: the interface never displays a fabricated distance, and the system announces loudly when it is running a placeholder model.

6.8 Limitations

Limitation	Consequence	Addressable by
Fall hazards not detected (`pothole` , `open_drain` zero recall)	**The system does not warn about the hazards that motivated it**	Manual annotation of 100–200 instances per class
Severe class imbalance (92% of boxes in two classes)	Rare classes contribute almost nothing to training	Targeted annotation; class-balanced sampling or loss weighting
No metric distance	Guidance says "close" and "far", never a distance	Monocular depth network or stereo sensor via `DepthProvider`
Bengali speech requires internet	Default language unavailable offline on this machine	Installing a Bengali system speech voice
No object tracking	Two people on the left are one item of news; objects not followed across frames	Adding a tracker (e.g. ByteTrack) keyed to detection identity
Daytime imagery only	Night and adverse-weather behaviour unknown	Collection under varied conditions
Low source resolution (256×256)	Limits detail for small and distant objects	Re-collection at higher resolution
Single monocular camera	No depth, no motion estimate, no orientation	Sensor fusion with IMU or depth
No user evaluation	Usefulness of the guidance is unmeasured	Study with visually impaired participants under ethical approval
Verification confirmed rather than corrected labels	Annotation errors from the machine stage persist in the dataset	A second review pass with correction as the explicit task

Table 6.1 — Summary of limitations and how each might be addressed.

6.9 Ethical and Safety Considerations

This system is a research prototype. It has not been validated with visually impaired users, has known and substantial detection failures, and must not be treated as a sole navigation aid. Any deployment would need to position it as supplementary to a cane or guide dog, never as a replacement.

The most serious ethical risk is not that the system fails, but that it fails *selectively* in a way a user cannot perceive. A user who hears reliable warnings about people and vehicles may reasonably infer that silence means a clear path, when in fact the system is structurally incapable of reporting an open drain. That asymmetry between apparent and actual competence is more dangerous than uniformly poor performance would be, and it is the reason the zero-recall results are stated so prominently in this thesis rather than being averaged into an overall figure.

Two further considerations apply. The dataset consists of public street scenes in which people appear incidentally; no individual was recruited or photographed as a subject, and the dataset has not been published. Bengali speech synthesis transmits the generated sentence to a third-party service, which is a privacy consideration if the system were deployed — the sentences describe the user's immediate surroundings. An offline voice would remove this exposure.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

This thesis set out to build an assistive system that detects footpath obstacles from a monocular camera and communicates them as spoken guidance in Bengali and English, using a detector trained on locally collected imagery rather than a general-purpose benchmark.

The system was built and works. A dataset of 2,782 annotated Dhaka footpath images was assembled from an unlabelled pool of 21,359 through a documented selection, de-duplication and machine-assisted annotation pipeline. A YOLO11n detector fine-tuned on it achieves mAP@0.5 of 0.419 and mAP@0.5:0.95 of 0.297 on a held-out test split, with AP@0.5 of 0.837 for `person` and 0.781 for `parked_vehicle`. Detections are converted into guidance by a deterministic layer — zone assignment, priority scoring, event-based announcement and cooldown — and rendered as grammatical Bengali or English speech. The same pipeline serves live camera, uploaded image and recorded video input at 22.9 and 13.7 FPS respectively.

The system does not, however, do the thing that most motivated it. `pothole` and `open_drain` obtain zero recall, because the annotation method that made the dataset feasible could not propose candidates for ground-surface defects, leaving 17 and 12 training instances respectively. The result is a working assistive prototype that is reliable on people and vehicles and blind to the fall hazards it was designed to warn about. Establishing that limitation clearly, and tracing it to a specific and correctable cause, is a legitimate outcome of the work — but it is a limitation, not a success.

7.2 Research Contributions

1. A documented, reproducible pipeline converting an unlabelled mixed-source pool into an annotated, de-duplicated footpath detection dataset, with leakage prevented at the split level.
2. An empirical characterisation of open-vocabulary annotation on Bangladeshi footpath imagery, with per-class evidence of where it transfers and where it fails — a result that is useful to anyone attempting a similar dataset.
3. A trained nine-class detector for this domain, with per-class results reported including the classes that fail.
4. A fully specified, non-learned navigation layer whose behaviour can be audited line by line, including an event-comparison detail (compare against last announcement, not last frame) that a naive implementation gets wrong.
5. A bilingual guidance generator handling Bengali verb-final order and animacy-dependent classifiers.
6. A single detection pipeline serving three input modes with measured performance for each.

7.3 Limitations

The limitations are set out in Table 6.1. The three that most constrain the current system are the absence of hazard-class detection, the absence of metric distance estimation, and the absence of any evaluation with visually impaired users. The first is a data problem with a clear remedy; the second requires an additional sensing or modelling component; the third requires a study design and ethical approval rather than engineering.

7.4 Future Work

Immediate, and necessary before any claim of hazard detection.

- Manually annotate 100–200 instances each of `pothole`, `open_drain`, `stairs` and `curb`. The unused image pool already contains a large number of pothole and stairs photographs, so the images exist; only the boxes are missing.
- Retrain and re-evaluate, reporting per-class AP for the hazard classes specifically.
- Apply class-balanced sampling or loss weighting so that rare classes contribute proportionally more gradient signal.

System capability.

- Integrate monocular depth estimation [18] through the existing `DepthProvider` interface, replacing apparent-size proximity with relative depth and allowing the system to speak in distance terms without fabricating them.
- Add object tracking so that guidance can refer to a specific object across frames and describe motion (approaching, receding) rather than only presence.
- Measure hazard-to-speech latency end to end, which is the metric that actually matters for the user and which frame rate does not capture.

Data and deployment.

- Extend collection to night, rain and artificial lighting, and to higher source resolution.
- Export to ONNX or NCNN for smartphone deployment, which is the realistic form factor for a pedestrian aid; the current laptop implementation is a development platform, not a product.
- Install or train an offline Bengali speech voice to remove the network dependence.

Evaluation.

- Conduct a user study with visually impaired participants under appropriate ethical approval, measuring whether instructions are judged timely, correct and sufficient. Without this, the navigation layer's usefulness remains asserted rather than demonstrated.

> **Implemented in this work:** nine-class footpath detection; twelve indoor classes via a pre-trained model; zone and proximity estimation; priority ranking; event-based announcement with cooldown; Bengali and English generation and speech; live camera, image and recorded video modes; browser interface.
>
> **Not implemented, and listed above as future work:** hazard-class detection; depth estimation; object tracking; GPS or IMU integration; mobile deployment; offline Bengali speech; user evaluation.

Verification note. The references below are standard, widely cited works in computer vision and assistive technology, cited for the specific claims attributed to them. Bibliographic details should be checked against the publisher's record before submission, and page numbers and DOIs added where the department's style requires them; they have been omitted here rather than reproduced from memory.

[1] World Health Organization, *World Report on Vision*. Geneva, Switzerland: World Health Organization, 2019.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[4] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *Proc. European Conf. Computer Vision (ECCV)*, 2016.

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017.

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint* arXiv:2004.10934, 2020.

[7] G. Jocher and J. Qiu, "Ultralytics YOLO11," Ultralytics, 2024. [Software]. Available: https://github.com/ultralytics/ultralytics

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014.

[9] R. Tapu, B. Mocanu, and T. Zaharia, "Wearable Assistive Devices for Visually Impaired: A State of the Art Survey," *Pattern Recognition Letters*, vol. 137, pp. 37–52, 2020.

[10] P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2001.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[12] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Proc. European Conf. Computer Vision (ECCV)*, 2020.

[13] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.

[14] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint* arXiv:1804.02767, 2018.

[15] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[16] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-Time Open-Vocabulary Object Detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[17] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models from Natural Language Supervision," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.

[18] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, 2022.

[19] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Appendix A — Dataset Information

Property	Value	
Initial image pool		21,359
Domain-selected (Dhaka footpath)		3,000
After near-duplicate removal		2,787
Annotated and exported		2,782
Used for training (paired, non-empty)		2,763
Total bounding boxes		23,962
Source image resolution	256 × 256	
Training resolution	640 × 640 (letterboxed)	
Annotation format	YOLO (`class_id cx cy w h` , normalised)	
De-duplication method	Difference hash, Hamming distance ≤ 5	
Split ratio / seed	70 / 20 / 10, seed 42, cluster-level	

Appendix B — Class List

Thesis classes (trained on project data) — see Table 3.1 for full definitions:

```
0  person          মানুষ              5  open_drain       খোলা ড্রেন
1  pole            খুঁটি              6  parked_vehicle   পার্ক করা গাড়ি
2  tree            গাছ                7  street_obstacle  রাস্তার প্রতিবন্ধকতা
3  stairs          সিঁড়ি             8  curb             ফুটপাতের কিনারা
4  pothole         গর্ত
```

Indoor classes (COCO-pretrained, runtime only, no thesis-measured mAP):

```
chair (চেয়ার)      couch (সোফা)        bed (বিছানা)         dining table (টেবিল)
tv (টেলিভিশন)      toilet (টয়লেট)     sink (সিংক)          refrigerator (ফ্রিজ)
oven (ওভেন)        microwave (মাইক্রোওয়েভ)   potted plant (গাছের টব)   bench (বেঞ্চ)
```

`door` is not supported: it is absent from the COCO category list and was not annotated in the thesis dataset.

Appendix C — Important Configuration

```
DETECTION
  confidence            0.50   (adjustable 0.10 - 0.95 at runtime)
  iou (NMS)             0.45   inference   /   0.60 evaluation
  imgsz                 640
  max_det               30
  device                auto (CUDA if available, else CPU)
  indoor_enabled        true
  indoor_conf           0.40

SPATIAL
  zone_left_edge        0.33
  zone_right_edge       0.66
  near_area_ratio       0.15
  medium_area_ratio     0.04

NAVIGATION AND VOICE
  language              bn  (default)
  announce_cooldown     3.0 s   per (class, position)
  global_min_gap        1.2 s   between any two utterances
  max_spoken_objects    3       combined into ONE sentence
  announce_far          false
  tts_backend           auto -> edge-tts (bn) / pyttsx3 (en)
  tts_voice_bn          bn-BD-NabanitaNeural

STREAM
  jpeg_quality          70
  stream_fps_cap        20
```

Appendix D — Algorithms

Pseudocode for the five core algorithms is given in Section 3.15 and is not repeated here.

Appendix E — Sample Detection Results

See Figures 5.6 and 5.7. Additional annotated outputs are stored in the project under `reports/figures/`.

Appendix F — User Interface Screenshots

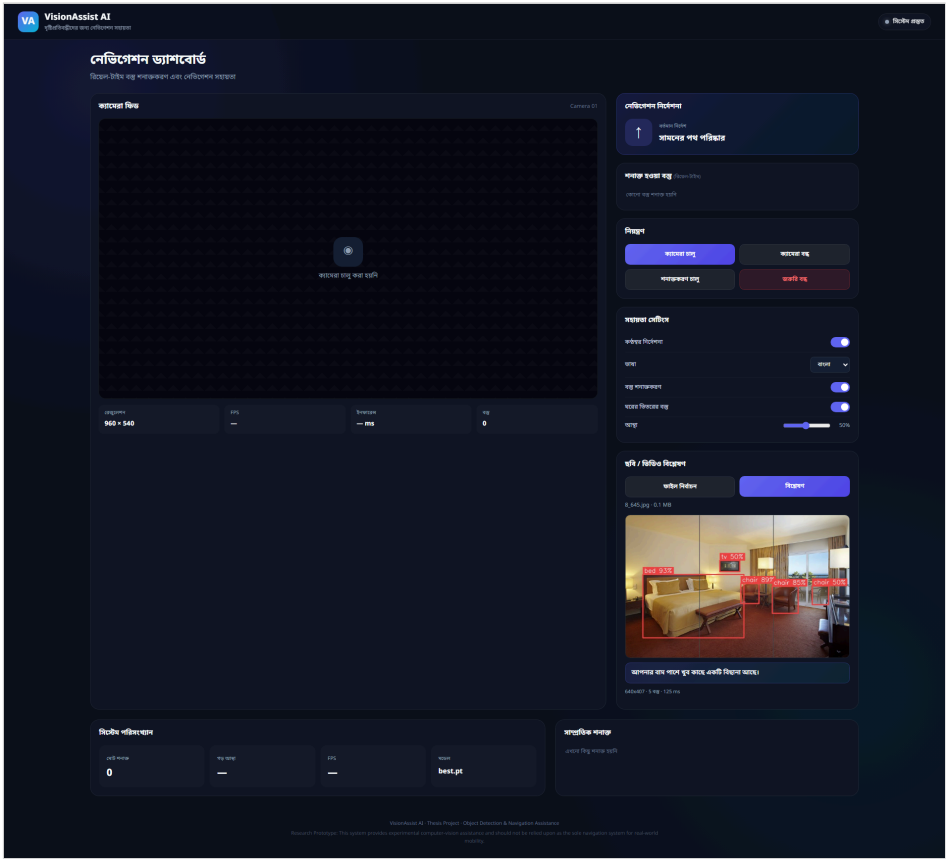


Figure F.1 — Interface in image mode, showing the annotated result, the generated Bengali instruction and the detection metadata.

Figure 5.9 shows the interface in recorded-video mode.

Appendix G — Experimental Tables

Overall and per-class results are in Tables 5.3 and 5.4; training progression in Table 5.2; runtime performance in Table 5.7; annotation transfer in Table 5.1. Raw outputs are stored in the project at `runs/detect/runs/evaluate/exp/metrics.json` and `runs/detect/runs/train/thesis_v1/results.csv`.

Appendix H — Installation and Execution

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Install the CUDA build of PyTorch (the default wheel is CPU-only)
pip install --upgrade --index-url https://download.pytorch.org/whl/cu128 torch torchvision
python -c "import torch; print(torch.cuda.is_available())"     # must print True

# 3. Run the application
python app/main.py --port 8010                # then open http://127.0.0.1:8010

# Useful options
python app/main.py --camera 1                 # different camera index
python app/main.py --camera clip.mp4          # video file as source
python app/main.py --language en              # start in English
python -m backend.camera                      # list working camera sources

# 4. Reproduce the dataset and model
python scripts/build_annotation_set.py        # select and de-duplicate
python scripts/autolabel.py                   # candidate annotations
python scripts/review_labels.py               # human verification UI
python scripts/prepare_dataset.py             # 70/20/10 splits
python scripts/train.py                       # train -> best.pt
python scripts/validate.py                    # P, R, mAP, confusion matrix
```

Appendix I — Selected Source Code

I.1 — Position and proximity classification (`backend/spatial.py`):

```
def get_position(self, center_x, frame_width):
    frac = center_x / float(frame_width)
    s = self.settings
    if frac < s.get("zone_left_edge"):
        return LEFT
    if frac > s.get("zone_right_edge"):
        return RIGHT
    return CENTER

def get_proximity(self, bbox, frame_width, frame_height):
    x1, y1, x2, y2 = bbox
    area_ratio = ((x2 - x1) * (y2 - y1)) / float(frame_width * frame_height)
    if self.depth_provider is not None:        # extension point for real depth
        metres = self.depth_provider.depth_at(...)
        if metres is not None:
            return (NEAR if metres <= 1.5 else
                    MEDIUM if metres <= 4.0 else FAR), area_ratio
    s = self.settings
    if area_ratio >= s.get("near_area_ratio"):
        return NEAR, area_ratio
    if area_ratio >= s.get("medium_area_ratio"):
        return MEDIUM, area_ratio
    return FAR, area_ratio
```

I.2 — Priority scoring (`backend/navigation/navigation_engine.py`):

```
def priority_score(self, det):
    score = 0.0
    if det["class_name"] in DANGER_CLASSES:
        score += 100.0
    elif det["class_name"] in INDOOR_OBSTACLES:
        score += 40.0
    if det.get("position") == "CENTER":
        score += 50.0
    score += {"NEAR": 30.0, "MEDIUM": 12.0, "FAR": 0.0}.get(
        det.get("proximity", "FAR"), 0.0)
    score += float(det.get("confidence", 0.0)) * 10.0
    return score
```

I.3 — Bengali sentence construction (`backend/voice/message_generator.py`):

```
def _generate_bn(object_name, position, proximity, urgent, overrides):
    name    = bengali_name(object_name, overrides)
    article = _bn_article(object_name, name)     # একজন for people, একটি otherwise
    place   = _BN_POSITION.get(position, "সামনে")
    prox    = _BN_PROXIMITY.get(proximity, "")

    # Subject-Object-Verb: [place] [proximity] [article] [noun] আছে।
    pieces = [place]
    if prox:
        pieces.append(prox)
    pieces.append(article)
    pieces.append(name)
    sentence = " ".join(pieces) + " আছে।"
    return "{} {}".format(_BN_WARN, sentence) if urgent else sentence
```

I.4 — Dual-model fusion (`backend/detector.py`):

```
@staticmethod
def _merge(primary, secondary, iou_thr=0.55):
    """Keep only secondary boxes that do not overlap a thesis-model box.
       The thesis model always wins, because it is trained on this data."""
    return [d for d in secondary
            if not any(iou(d, p) > iou_thr for p in primary)]
```

> VisionAssist AI — Object Detection and Navigation Assistance for Visually Impaired Individuals. Research prototype. This system provides experimental computer-vision assistance and must not be relied upon as the sole navigation aid for real-world mobility.